Bilkent University
Department of Computer Engineering
CS491/2 Senior Design Project
T2517
Moo-tion
Final Report

Ahmet Burak Kılıçkaya	22103833	burak.kilickaya@ug.bilkent.edu.tr
Canberk Özba	22101815	canberk.ozba@ug.bilkent.edu.tr
Emir Eroğlu	22202233	emir.eroglu@ug.bilkent.edu.tr
Mert Özkaya	22002460	mert.ozkaya@ug.bilkent.edu.tr
Muhammed Enes Şahin	22102919	enes.sahin@ug.bilkent.edu.tr

Supervisor: Doruk Öner

Course Instructors: Mert Bıçakçı, İlker Burak Kurt

01.05.2026

This report is submitted to the Department of Computer Engineering of Bilkent University in partial fulfillment of the requirements of the Senior Design Project course CS491/2.

1. Introduction **4**
1.1. Project Motivation and Background 4
1.2. Project Aim and Objectives 4
1.4. Project Significance and Alternative Solutions 6
2. Requirements Details **6**
2.1. Functional Requirements 6
2.1.1. Account and Farm Management 7
2.1.2. Real-Time Vision and Detection 7
2.1.3. Herd and Lifecycle Tracking 7
2.1.4. Alerting and Notifications 7
2.1.5. Hardware Management 8
2.2. Non-Functional Requirements 8
2.2.1. Usability 8
2.2.2. Performance 8
2.2.3. Reliability 8
2.2.4. Scalability 9
2.2.5. Supportability 9
2.2.6. Security 9
2.2.7. Maintainability 9
2.3. Pseudo Requirements 9
3. Final Architecture and Design Details **10**
3.1. High-Level Architectural Overview and Design Philosophy 10
3.2. Detailed Subsystem Design 11
3.2.1. The Client Subsystem (Presentation Layer) 11
3.2.2. The Central Application Gateway (API Layer) 12
3.2.3. The Vision Processing Engine (AI Worker Layer) 14
3.3. Hardware and Software Mapping 15
3.4. Hybrid Persistent Data Management 16
3.5. Access Control and Security Mechanisms 17
3.6. Algorithmic Event Processing and Deduplication Design 18
4. Development and Implementation Details **20**
4.1. Methodological Approach and Development Lifecycle 21
4.2. Artificial Intelligence and Vision Pipeline Implementation 21
4.2.1. Dataset Curation and Model Fine-Tuning 21
4.2.2. Real-Time Inference and Algorithmic Deduplication 22
4.3. Central Application Gateway and Backend Implementation 22
4.3.1. Security Middleware and Identity Verification 22
4.3.2. The Reproductive State Machine 23
4.4. Client Application and User Interface Implementation 23
4.4.1. Reactive State Management Implementation 23
4.4.2. Cross-Platform Video Rendering Pipeline 24

4.5. Infrastructure, Containerization, and Deployment..... 24
5. Test Cases and Results.....24
5.1 Functional Test Cases..... 25
5.1.1 User Account and Authentication Test Cases.....25
5.1.2 Farm and User Management Test Cases..... 32
5.1.3 Camera Management Test Cases..... 34
5.1.4 Vision Pipeline and Detection Test Cases..... 37
5.1.5 Notification and Alert Test Cases.....40
5.1.6 Integration Test Cases..... 40
5.2 Non Functional Test Cases.....45
5.2.1 Performance Test Cases.....45
5.2.2 Usability Test Cases.....47
5.2.3 Reliability Test Cases.....48
5.2.4 Scalability Test Cases.....51
5.2.5 Compatibility Test Cases.....54
6. Maintenance Plan and Details..... 55
6.1. Infrastructure and Relational Data Maintenance..... 55
6.1.1. Automated Database Schema Migrations..... 56
6.1.2. Cloud Storage Retention and Archival Policies..... 56
6.2. Artificial Intelligence and Active Learning Maintenance.....56
6.2.1. The Active Learning Feedback Loop..... 56
6.2.2. Periodic Model Fine-Tuning and Re-Deployment..... 57
6.3. Hardware Ecosystem and Network Support.....57
6.3.1. Automated Camera Health Polling.....57
6.3.2. Visual Degradation Monitoring.....57
7. Other Project Elements..... 57
7.1. Consideration of Various Factors in Engineering Design..... 58
7.1.1. Engineering and Environmental Constraints.....58
7.1.2. Economic and Social Factors..... 58
7.1.3. Interoperability and Regulatory Standards.....59
7.2. Ethics and Professional Responsibilities..... 59
7.3. Teamwork Details..... 60
7.3.1. Contributing and Functioning Effectively.....60
7.3.2. Helping Create a Collaborative and Inclusive Environment.....61
7.3.3. Taking Lead Role and Sharing Leadership.....61
7.3.4. Meeting the Project Objectives..... 62
7.4. New Knowledge Acquired and Applied.....62
8. Conclusion and Future Work.....63
8.1. Project Conclusion.....63
8.2. Summary of Key Achievements.....63
8.3. Future Work and System Expansion..... 64

8.3.1. Automated Animal Re-identification....64
8.3.2. Comprehensive Holistic Behavioral Analytics....65
8.3.3. Offline Edge Computing Infrastructure....65
8.3.4. Hardware Ecosystem and Facility Integration....65
9. Glossary.... 66
Moo-tion User Manual.... 70
1 - Logging into Moo-tion.... 70
1.1 - Sign-up.... 70
1.2 - Sign-in....71
2 - Moo-tion Homepage and Notifications.... 72
2.1 - Home Dashboard....72
2.2 - Notifications Panel....72
3 - Farm Management.... 73
3.1 - Managing Existing Farms.... 73
3.2 - Creating a New Farm....74
4 - Herd and Cow Lifecycle Operations.... 75
4.1 - Herd Overview.... 75
4.2 - Adding and Managing Cow Profiles....76
5 - Detection Management (Tespitler)....78
5.1 - Reviewing New Detections.... 78
5.2 - Approved and Deleted Queues.... 80
6 - Camera Management....82
6.1 - Camera Health and Monitoring....82
6.2 - Connecting and Editing Cameras.... 83
7 - Settings, Account, and Preferences.... 85
7.1 - General System Settings.... 85
7.2 - Account Profile and Notification Channels....86
10. References.... 89

1. Introduction

1.1. Project Motivation and Background

The global agricultural sector, and more specifically the commercial dairy farming industry, operates within highly constricted economic margins where operational efficiency, precise resource management, and biological timing inextricably dictate overall profitability. One of the most historically persistent, labor-intensive, and economically significant challenges facing modern dairy facility managers is the accurate and timely detection of estrus, commonly referred to in agricultural literature as the heat cycle in cattle. Biological reproduction serves as the fundamental catalyst for all dairy production; a cow must successfully conceive and give birth to initiate a new lactation phase. Consequently, the financial stability of a dairy enterprise is directly proportional to the reproductive efficiency of its herd.

The physical and behavioral markers of estrus are frequently subtle, highly transient, and heavily skewed toward the late-night and early-morning hours when human supervision within the barn is either minimal or absent. Primary behavioral indicators, such as the acceptance of mounting by other cattle, elevated restlessness, and distinct physical interactions, often present themselves within a narrow biological window lasting only a few hours. When a dairy farmer or herdsman misses a cow’s critical estrus window, artificial insemination must be postponed until the onset of the subsequent reproductive cycle, which typically occurs twenty-one days later.

This systemic delay manifests economically as "extended non-productive days," a costly period during which the animal continues to consume feed, water, and baseline veterinary resources without contributing to the facility's overall milk yield. Across a large-scale commercial herd containing hundreds or thousands of cattle, the cumulative economic degradation caused by missed estrus cycles is staggering. Traditional agricultural methodologies rely almost exclusively on continuous manual observation by trained herdsmen to mitigate this issue. However, this approach is fundamentally vulnerable to human error, worker fatigue, the practical limitations of nighttime visibility, and the increasing global shortage of skilled agricultural labor.

1.2. Project Aim and Objectives

The primary objective of the Moo-tion project is to engineer, validate, and deploy a highly scalable, intelligent monitoring platform that fundamentally transforms standard barn surveillance footage into actionable, precise reproductive insights. By leveraging the synthesis of advanced computer vision algorithms, distributed cloud computing infrastructure, and cross-platform mobile accessibility, the system is designed to achieve three foundational engineering objectives.

First, the project aims to fully automate the continuous detection of specific behavioral markers associated with the bovine reproductive cycle, completely removing the reliance on human visual observation. This involves the development of an isolated vision processing pipeline capable of ingesting live network video streams and accurately identifying overlapping

geometric interactions between animals in real-time, regardless of challenging environmental lighting or physical occlusions common in agricultural settings.

Second, the system seeks to establish a centralized, secure digital management interface for comprehensive herd records, historical reproductive timelines, and hardware configurations. This objective ensures that fragmented paper records and isolated facility spreadsheets are replaced with a secure, multi-tenant relational data structure that maintains absolute referential integrity over the lifespan of the herd.

Third, the project is designed to engineer a real-time, multi-channel notification workflow that seamlessly bridges the operational gap between algorithmic anomaly detection and immediate human intervention. The objective is not to replace the farmer, but rather to function as an advanced decision-support tool, providing human operators with verified visual evidence of a biological event so that timely, highly informed veterinary actions can be executed.

1.3. Target Audience

The technological ecosystem developed within the Moo-tion project is tailored for a specific, multi-tiered demographic within the agricultural industry. The primary target audience consists of commercial dairy farm owners and facility managers who require macroscopic oversight of their operational efficiency and hardware infrastructure. For these stakeholders, the platform provides critical data regarding herd fertility rates and system-wide monitoring capabilities.

The secondary target audience comprises on-site agricultural laborers, herdsmen, and specialized caretakers. These individuals are the primary end-users of the mobile application interface. They require instantaneous, highly localized alerts regarding specific animals exhibiting reproductive behaviors, allowing them to optimize their daily workflow, locate the specific animal within the facility, and perform necessary biological interventions.

Furthermore, the system is designed to be highly beneficial to attending veterinarians and animal breeding specialists. By providing an immutable, historically accurate digital log of all reproductive events, visual evidence of physical behaviors, and automated calculations of projected gestation periods, the platform supplies veterinary professionals with the precise longitudinal data required to diagnose underlying fertility issues or optimize herd genetics.

1.4. Project Significance and Alternative Solutions

The contemporary agricultural technology market offers various commercial solutions aimed at mitigating the estrus detection problem. However, these legacy systems, often categorized under the umbrella of the Internet of Things, typically demand the procurement, installation, and ongoing maintenance of expensive and highly invasive physical hardware. The most prevalent solutions require the attachment of wearable biometric tracking collars, the

insertion of specialized radio-frequency identification ear tags, or the oral administration of heavy, ingestible tracking boluses that reside permanently within the animal's digestive tract.

These physical devices present several significant drawbacks. They are highly prone to environmental degradation from moisture and physical impact within the barn. They require frequent, labor-intensive battery replacements, which subjects the animals to unnecessary physical stress and handling. Most critically, outfitting a large commercial herd with individual physical sensors requires an immense, often prohibitive, initial capital expenditure.

Moo-tion introduces a profound paradigm shift within this technological landscape by providing a purely non-invasive, software-centric alternative. By applying state-of-the-art computer vision methodologies to standard network camera streams, the platform empowers agricultural operators to leverage their preexisting barn surveillance infrastructure. Rather than monitoring the biological metrics of a single cow via a wearable sensor, a single network camera running the Moo-tion detection algorithms can simultaneously monitor dozens of animals within its field of view.

This software-defined approach significantly reduces initial deployment costs, entirely mitigates hardware depreciation and battery maintenance concerns, and inherently prioritizes animal welfare by eliminating the physical burden of tracking apparatuses. The significance of this project lies in its ability to democratize access to high-tier precision agriculture, offering sophisticated artificial intelligence capabilities to farms that may lack the capital to invest in invasive physical sensor networks.

2. Requirements Details

The architectural and operational requirements for the Moo-tion platform were formalized following an exhaustive analysis of the daily operational realities inherent to commercial dairy farm management. These practical agricultural necessities were subsequently balanced against the strict computational constraints and network limitations dictated by real-time video processing and distributed machine learning inference. The resulting system requirements are classified into functional requirements, which define the specific behavioral capabilities the system must possess, and non-functional requirements, which dictate the systemic quality attributes, performance thresholds, and security standards the architecture must uphold.

2.1. Functional Requirements

The functional requirements focus on providing agricultural users with essential features for secure account management, real-time biological event detection, comprehensive herd tracking, and hardware monitoring. The system adapts to the dynamic nature of dairy farms to provide actionable decision support.

2.1.1. Account and Farm Management

- Users should be able to securely create accounts, log in, and reset passwords utilizing a secure cloud identity provider.
- Users should be able to create, manage, and seamlessly switch between multiple independent farm environments.
- Farm owners should be able to invite new workers to their farm and assign specific operational roles, such as manager or caretaker.
- Users should be restricted from accessing or viewing any hardware or animal data belonging to a farm they are not explicitly authorized to view.

2.1.2. Real-Time Vision and Detection

- The system should be able to continuously ingest and decode live network video streams from barn surveillance cameras.
- The background processing engine should automatically recognize and interpret specific physical mounting behaviors indicative of estrus.
- The system should computationally filter out false positives and deduplicate repetitive detections of the same biological event to prevent alert fatigue.
- The system should capture and digitally annotate a high-resolution evidence image for every confirmed biological event.

2.1.3. Herd and Lifecycle Tracking

- Users should be able to register individual cattle, recording their unique identification numbers, breed, and birth dates.
- Users should be able to manually log specific veterinary events, such as artificial inseminations or pregnancy confirmations.
- The system should automatically calculate projected gestation timelines and update the animal's reproductive status based on logged veterinary events.
- Users should be able to filter the herd roster by current reproductive status to quickly identify animals requiring attention.

2.1.4. Alerting and Notifications

- Users should receive real-time push notifications on their mobile devices whenever a verified estrus event is finalized by the processing engine.
- Users should be able to view a chronological queue of pending alerts, complete with the corresponding visual evidence.
- Users should be able to explicitly confirm or reject an alert, thereby linking the event to a specific animal's historical record.
- Users should be able to configure their individual notification preferences, including the ability to temporarily mute alerts.

2.1.5. Hardware Management

- Administrators should be able to register new surveillance cameras by providing a standard network streaming address.
- Users should be able to view live video feeds from their registered cameras directly within the mobile and web interfaces.
- The system should monitor the connectivity status of registered hardware and visually indicate if a camera goes offline.

2.2. Non-Functional Requirements

The non-functional requirements emphasize usability in agricultural environments, high reliability, processing performance, and strict data security. Data consistency, timely alert delivery, and computational resource efficiency must be maintained throughout the platform.

2.2.1. Usability

Moo-tion operates within a physically demanding agricultural environment. Therefore, the application interface must be highly intuitive, utilizing high-contrast visuals, clear iconography, and large touch targets suitable for users wearing protective gloves. Complex algorithmic metrics must be abstracted into recognizable farming terminology. The application must natively support multiple languages, specifically Turkish and English, to accommodate a diverse agricultural workforce without causing operational confusion.

2.2.2. Performance

The platform offers real-time object detection on high-definition video, which is highly computationally intensive. We must ensure that the heavy vision processing workloads are strictly isolated from the central application gateway. The system must minimize latency; the time elapsed between a physical event occurring in the barn and a notification arriving on the user's device must be kept to an absolute minimum. To preserve server resources, the video processing engine should utilize efficient frame-sampling techniques rather than analyzing every individual frame of the video feed.

2.2.3. Reliability

Dairy farms frequently experience unstable rural internet connections and power fluctuations. To ensure operational continuity, Moo-tion must possess robust fault-tolerance mechanisms. If a camera feed unexpectedly drops, the system must automatically initiate a reconnection sequence utilizing an exponential backoff strategy rather than failing silently. Furthermore, absolute data consistency must be maintained; a logged insemination event must instantly reflect across all authorized users' devices to prevent duplicate veterinary procedures.

2.2.4. Scalability

As agricultural operations expand, the system must scale seamlessly. The distributed architecture should allow facility managers to add dozens of new surveillance cameras without degrading the responsiveness of the mobile application. The platform must utilize a modular approach, allowing the underlying infrastructure to scale the background processing workers horizontally as the total volume of ingested video data increases across multiple independent farm deployments.

2.2.5. Supportability

To ensure the long-term viability of the platform, the system architecture must be highly supportable. The use of cloud-based relational data storage ensures that user settings and herd records remain perfectly synchronized across diverse hardware devices. The codebase must be systematically containerized to guarantee that deployment environments remain consistent, allowing future engineering teams to smoothly release necessary updates, expanded language support, or additional behavioral models without disrupting ongoing farm operations.

2.2.6. Security

Because Moo-tion continuously processes surveillance footage of private agricultural workplaces, strict security standards are paramount. All communication between the client devices and the central server must be securely encrypted. Access to farm data must be rigidly enforced through cryptographically signed authorization tokens. To protect the privacy of the farm's laborers, the system must implement strict data minimization; the platform must only save video frames containing a confirmed biological event, immediately discarding all other routine workplace footage from memory.

2.2.7. Maintainability

To support future growth and integration with other agricultural technologies, the system must be highly maintainable. The backend services and the mobile application must communicate through strictly defined, well-documented structural contracts to minimize the risk of breaking changes during updates. Automated database migration protocols must be established to ensure that future feature expansions can be applied to the relational database without risking the corruption of historical herd data.

2.3. Pseudo Requirements

In addition to the core functional and non-functional requirements, several external standards, legal frameworks, and industry practices were considered to ensure the system's ethical operation and commercial viability.

- The application must fully comply with workplace privacy laws, specifically the Personal Data Protection Law (KVKK) and the General Data Protection Regulation (GDPR),

ensuring video data is strictly access-controlled, and farm employees are protected from unwarranted continuous surveillance.

- The video ingestion pipeline must strictly adhere to standard international real-time streaming protocols to ensure plug-and-play compatibility with preexisting commercial surveillance hardware, avoiding proprietary vendor lock-in.
- The mobile application interface must align with established mobile design guidelines to provide a predictable, user-friendly experience across different device ecosystems.
- The system should prioritize the ethical treatment of animals by operating purely as a non-invasive decision-support tool, ensuring that human verification is always required before any physical veterinary intervention is executed.
- The application must fulfill all security and performance requirements mandated by major mobile application storefronts to ensure successful public publication and distribution.

3. Final Architecture and Design Details

To successfully satisfy the demanding performance, concurrency, and security requirements delineated in the previous section, the Moo-tion platform was engineered utilizing a highly decoupled, modular distributed systems architecture. This architectural paradigm was chosen specifically to ensure that the computationally exhaustive processes inherent to artificial intelligence inference do not degrade the rapid response times required by the central application interface. The resulting ecosystem is structured around four primary, independently scalable architectural pillars: the presentation layer, the central application gateway, the vision processing engine, and the hybrid persistence infrastructure.

3.1. High-Level Architectural Overview and Design Philosophy

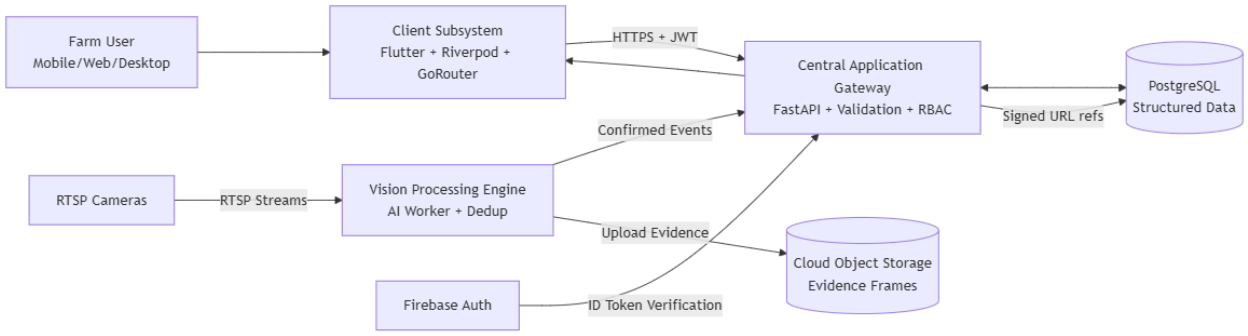


Figure 1. High-Level System Architecture and Design Philosophy

The fundamental design philosophy driving the Moo-tion architecture is the strict separation of concerns through component isolation. In traditional monolithic software applications, all operational logic, including data validation, database queries, and background computational tasks, resides within a single executable process. However, processing continuous, high-definition network video streams requires immense graphical and central

processing resources. If this processing were coupled with the web server responsible for serving data to mobile users, the entire system would experience catastrophic latency, rendering the application unusable during periods of high agricultural activity.

To circumvent this, the architecture entirely decouples the heavy video ingestion and inference workflows from the standard business logic. The system operates asynchronously; the vision processing engine acts as an independent producer of biological insights, while the central application gateway acts as a consumer and distributor of those insights. This strict boundary allows facility administrators to horizontally scale the processing hardware to accommodate hundreds of newly installed barn cameras without necessitating any structural changes to the client application or the relational database.

3.2. Detailed Subsystem Design

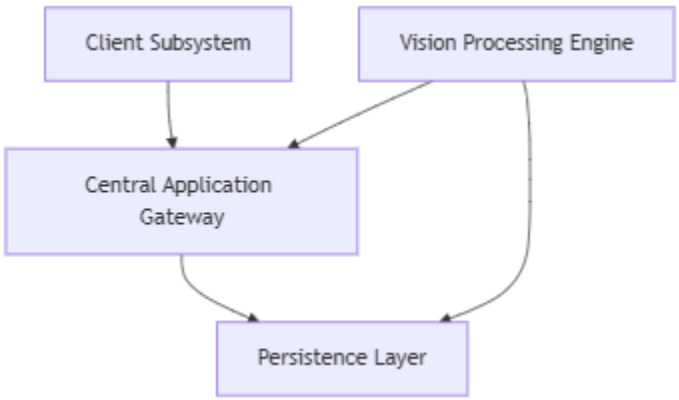


Figure 2. Subsystem Decomposition of the Moo-tion Platform

3.2.1. The Client Subsystem (Presentation Layer)

The presentation layer is constructed as a unified, single-codebase application capable of compiling natively for diverse mobile operating systems as well as modern desktop web browsers. This layer utilizes a reactive, declarative state management paradigm. In this model, the user interface does not actively fetch data; rather, it continuously listens to a localized, strictly managed data cache and redraws itself automatically whenever the underlying data mutates.

The user's currently selected farm environment acts as the absolute primary data filter for the entire application state. Transitioning between disparate farm environments triggers a deliberate, global state invalidation sequence. This mechanism forces all downstream interface components from the hardware dashboard to the individual animal health rosters to completely purge their cached data from the device's volatile memory and securely request fresh,

tenant-scoped information from the backend. This strict client-side isolation mirrors the backend security protocols, inherently preventing accidental cross-tenant data leakage on the user's personal device.

The client communicates with the central server exclusively via a secure, encrypted communication channel, utilizing standardized data transfer objects to ensure strict contract adherence. This ensures that the mobile application remains entirely agnostic to the underlying complexities of the backend infrastructure.

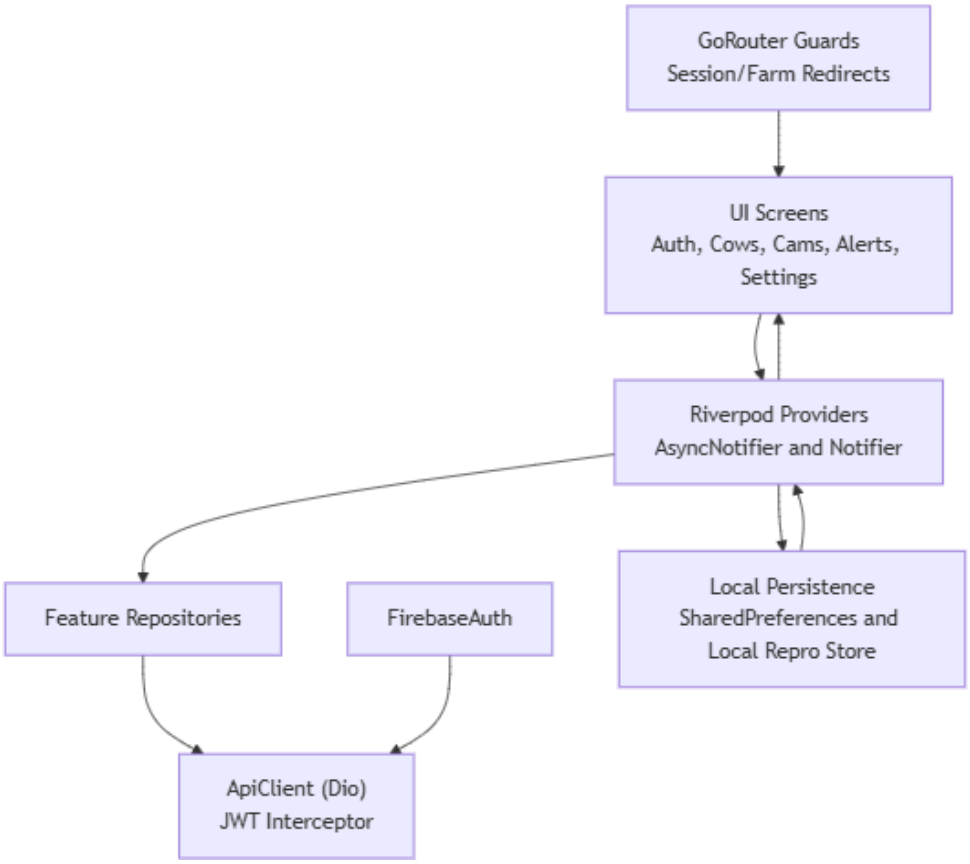


Figure 3. Client Subsystem (Presentation Layer) Architecture

3.2.2. The Central Application Gateway (API Layer)

Operating as the authoritative orchestrator of the entire digital ecosystem, the central application gateway handles all synchronous business logic, secure transactions with the relational persistence layer, and external cloud service coordination. This layer is constructed utilizing a highly asynchronous framework designed to manage thousands of concurrent input-output operations simultaneously without blocking execution threads.

A critical design feature of this gateway is its hierarchical validation middleware. Before any incoming data request is processed by the core business logic, it must pass through a strict, multi-tiered security and structural validation checkpoint. First, the cryptographic authorization token is verified for authenticity and expiration. Second, the user's role is cross-referenced against the requested action to ensure sufficient administrative privileges. Finally, the structural integrity of the incoming data payload is mathematically validated against predefined entity schemas, instantly rejecting any malformed or potentially malicious requests.

Furthermore, this layer houses the complex reproductive state machine. This algorithmic engine evaluates an animal's current biological status based on a chronological history of veterinary events and user inputs, automatically transitioning the animal through sequential phases of the reproductive lifecycle.

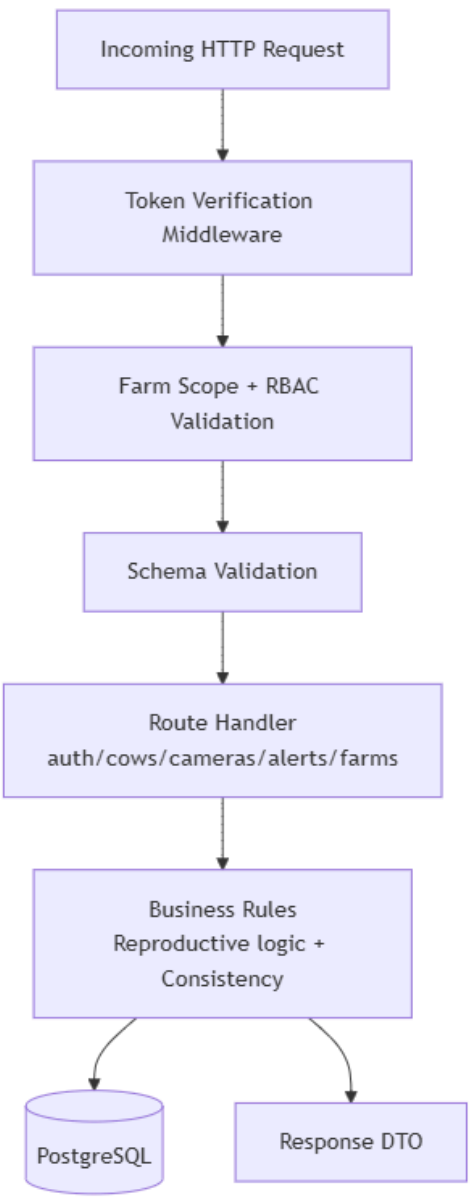


Figure 4. Central Application Gateway (API Layer) Request Lifecycle

3.2.3. The Vision Processing Engine (AI Worker Layer)

The vision processing engine is the computational workhorse of the Moo-tion platform. Operating completely independent of the application gateway, this subsystem manages the simultaneous ingestion, decoding, and analysis of dozens of continuous network video streams.

To prevent systemic conflicts within a distributed deployment environment, the engine utilizes a sophisticated lease-based concurrency control mechanism. When multiple worker instances are deployed across different physical servers, they must coordinate to ensure that two identical workers do not attempt to process the exact same barn camera simultaneously, which would result in duplicated alerts and wasted computational resources. Before analyzing a stream, a worker instance explicitly requests a time-limited cryptographic lock on that specific hardware identifier from the central database. If the lock is successfully acquired, the worker proceeds with ingestion; if the lock is held by another instance, the worker moves to the next available stream in the queue.

Once a stream is acquired, the engine applies a deliberate frame-sampling strategy. Rather than analyzing every sequential frame of a thirty-frame-per-second video, which is computationally wasteful given the relative physical speed of cattle, the engine extracts only a calculated fraction of frames per second. These raw video frames are then dynamically downscaled in memory to exactly match the mathematical resolution dimensions upon which the core object detection model was originally trained, drastically accelerating the inference speed.

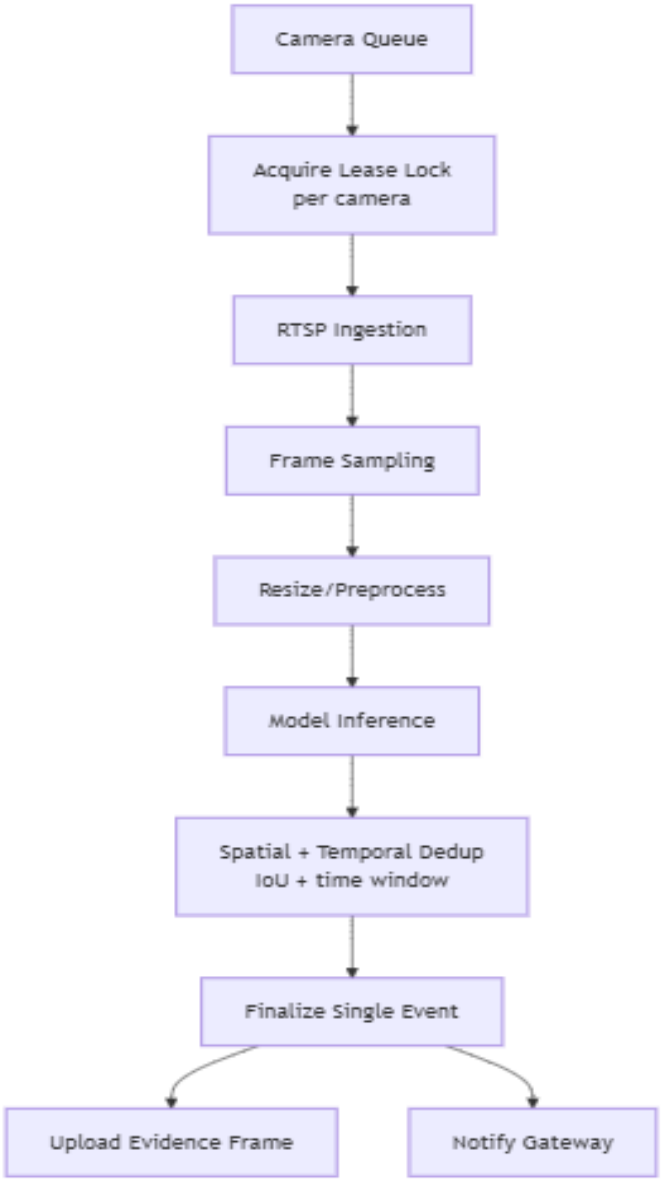


Figure 5. Vision Processing Engine (AI Worker Layer) Pipeline

3.3. Hardware and Software Mapping

While the core intelligence of the platform resides in the software, its physical deployment maps across a distinct topology of hardware nodes to bridge the physical barn environment with the cloud infrastructure.

- **The Sensory Edge (Hardware):** The physical edge consists of the preexisting network-connected surveillance cameras installed throughout the dairy facility. These devices broadcast continuous raw video data across the local agricultural network

utilizing standard real-time streaming protocols. They act purely as data emitters; no proprietary software is installed directly onto the camera hardware, preserving plug-and-play interoperability.

- **The Processing Core (Software/Server):** The central application gateway, the vision processing engine, and the persistent data management systems are mapped to high-performance, virtualized server hardware. The vision processing software is explicitly mapped to server nodes equipped with dedicated graphical processing units, providing the necessary mathematical acceleration required for real-time neural network inference.
- **The Interactive Edge (Hardware/Software):** The presentation subsystem is mapped directly to the personal hardware of the farm personnel, encompassing smartphones, tablet computers, and administrative desktop terminals.

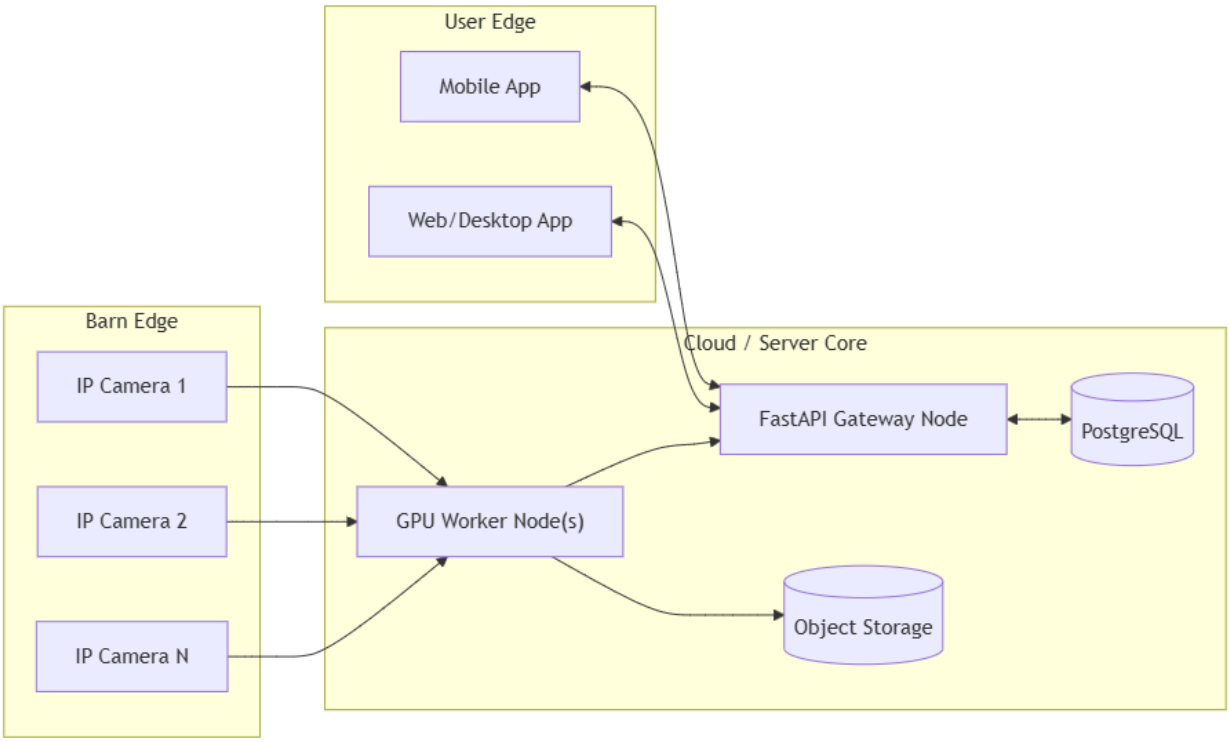


Figure 6. Hardware–Software Deployment Mapping

3.4. Hybrid Persistent Data Management

The platform employs a bifurcated data persistence strategy, routing data to distinct storage mechanisms based on its structural nature and access frequency.

Highly structured operational data, including user profiles, complex multi-tenant membership mappings, hardware network configurations, and longitudinal animal health records, is persisted within a robust relational database. The relational schema is designed with strict foreign key constraints and aggressive indexing, ensuring that complex analytical queries

execute with near-instantaneous rapidity, even as historical alert records compound over decades of continuous platform usage.

Conversely, unstructured media, specifically the high-resolution, annotated images captured by the vision engine that serve as visual evidence for generated alerts, are offloaded to a dedicated, highly available cloud object storage service. When a visual event is confirmed, the image is uploaded directly to this cloud repository, and only a lightweight, cryptographically signed universal resource locator is passed back to the relational database. This hybrid approach guarantees that the relational database remains lightweight, optimized, and incredibly fast, while massive media files are handled by infrastructure specifically architected for rapid global content delivery.

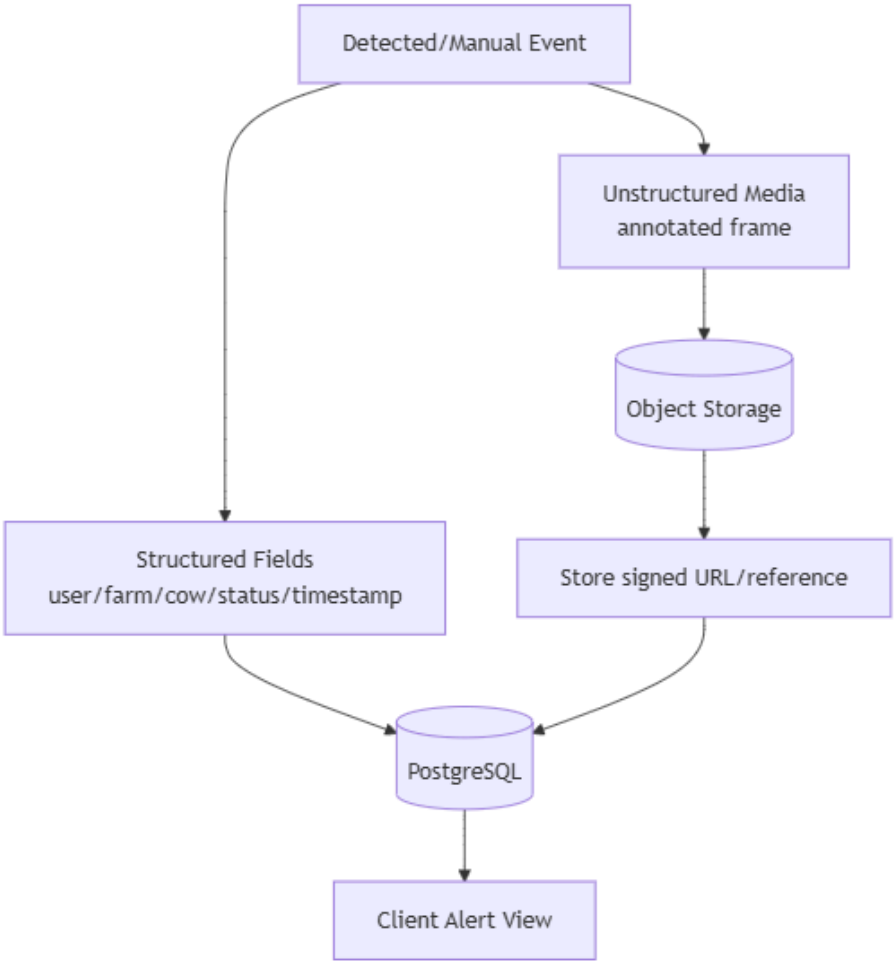


Figure 7. Hybrid Persistent Data Management Model

3.5. Access Control and Security Mechanisms

Given that the system inherently processes continuous video feeds of private agricultural workplaces and contains proprietary herd fertility data, the architecture necessitates a highly specialized, multi-layered access control framework.

- **Cryptographic Identity Verification:** The system abstracts core identity verification to a specialized third-party cloud authentication provider. When a user successfully authenticates, a time-limited, asymmetrically signed cryptographic token is issued. The gateway's security middleware mathematically verifies this token's signature for every request.
- **Multi-Tenant Data Isolation:** The platform safely hosts multiple independent agricultural entities on the exact same underlying server infrastructure. Access control is enforced at the lowest possible data query layer. Every operational data record is inextricably linked to a unique farm identifier. The middleware intercepts every incoming request, guaranteeing that a user can only retrieve or mutate data explicitly tagged with their authorized farm identifier.
- **Hierarchical Role-Based Access Control:** Within each isolated farm environment, the system implements granular role-based access control. The security middleware evaluates the user's specific role against the requested action. Destructive commands, such as deleting a camera configuration or permanently archiving an animal record, are actively prevented for standard caretakers and reserved strictly for facility owners.

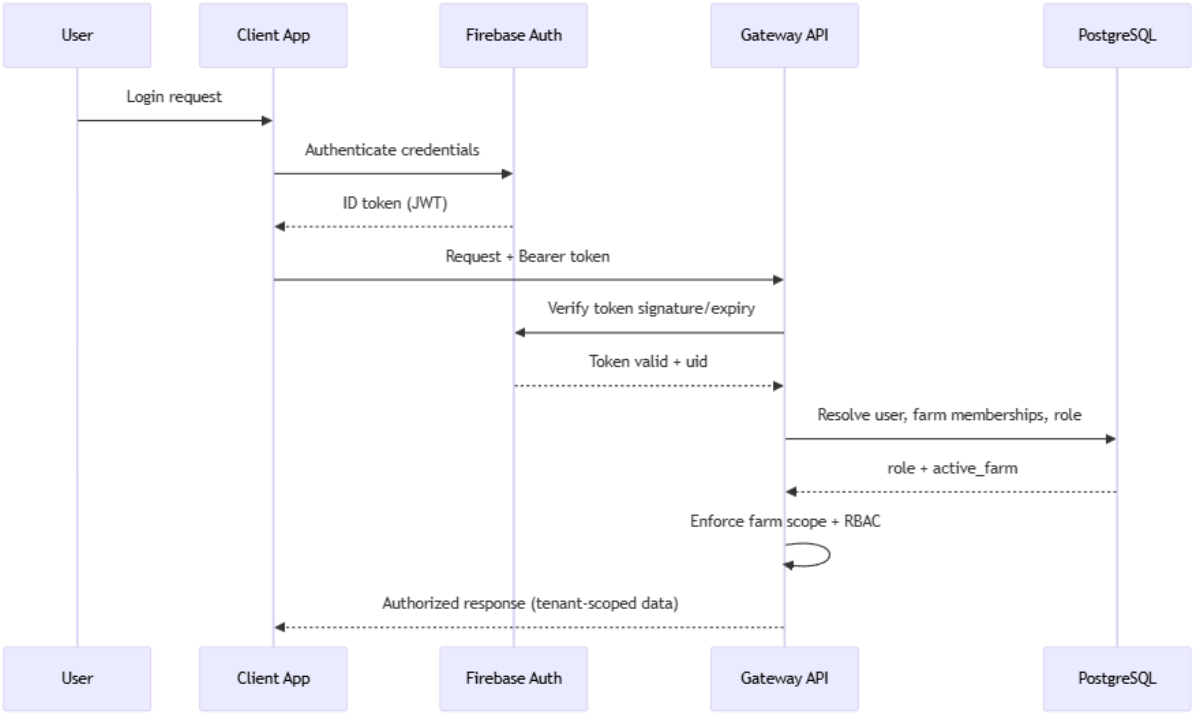


Figure 8. Access Control and Security Mechanism

3.6. Algorithmic Event Processing and Deduplication Design

The most mathematically complex design element within the architecture is the algorithmic process responsible for translating raw artificial intelligence inferences into actionable, singular human alerts.

Because a physical mounting event indicative of estrus spans several seconds, a frame-by-frame analysis will naturally generate dozens, if not hundreds, of independent positive detections for a single real-world biological incident. If the system were to dispatch a push notification for every positive inference, the farm administrator's mobile device would be overwhelmed with redundant alerts, leading to immediate system fatigue and ultimate abandonment.

To solve this, the architecture features a continuous spatial and temporal deduplication engine. As the object detection model identifies relevant subjects, it outputs specific geometric bounding boxes encompassing the subjects within the two-dimensional frame. The deduplication engine temporarily caches these geometric coordinates alongside their precise temporal timestamps.

When a subsequent frame is analyzed milliseconds later, the engine calculates the intersection over union, the exact percentage of spatial overlap between the newly generated bounding boxes and the cached coordinates from the preceding frames. If the spatial overlap exceeds a strict, predefined mathematical threshold, the engine confidently deduces that these individual detections represent a single, sustained, continuous biological event. The engine computationally merges these repeated occurrences, continually updating the event's overall duration rather than generating a new distinct record.

Only when the spatial overlap ceases, indicating that the physical interaction has concluded, does the engine finalize the historical record, securely upload the single most confident visual evidence frame to the cloud repository, and signal the central application gateway to dispatch a consolidated push notification to the farm personnel. This elegant deduplication logic is the cornerstone of the platform's usability, ensuring that the technology operates silently in the background, demanding human attention only when a verified, cohesive anomaly has been fully processed and confirmed.

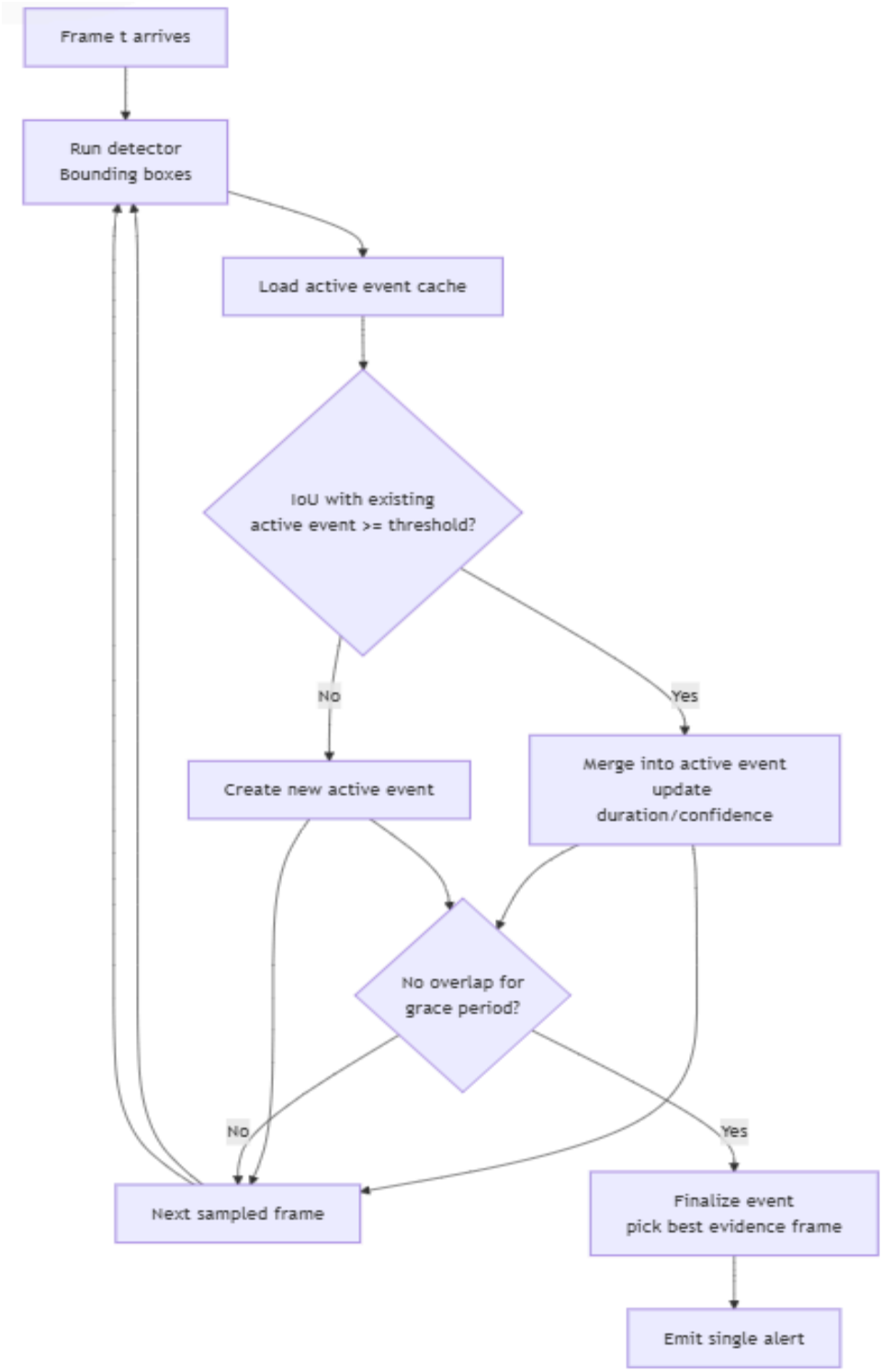


Figure 9. Event Deduplication Algorithm Flow

4. Development and Implementation Details

The transition from theoretical architectural design to a functional, production-ready software ecosystem required a disciplined, highly structured approach to software engineering. The implementation phase of the Moo-tion platform was characterized by rigorous project management, complex algorithmic optimization, and the meticulous translation of biological domain knowledge into executable computational logic. This section details the specific methodologies, technical hurdles, and engineering solutions employed during the construction of the system's four primary subsystems.

4.1. Methodological Approach and Development Lifecycle

To successfully navigate the complexities of building a distributed, multi-disciplinary system, the engineering team adopted a highly structured Agile development methodology, operating within strict, two-week iterative sprint cycles. This framework was selected specifically to mitigate the integration risks inherent to projects requiring simultaneous development across artificial intelligence, backend infrastructure, and mobile interfaces.

The implementation phase commenced with a practice known as contract-driven development. Before a single line of executable logic was written, the frontend and backend engineering sub-teams collaboratively established exhaustive, mathematically precise data exchange contracts. These contracts defined the exact structural schemas, expected data types, and required security headers for every single transaction that would occur between the client device and the central server.

By finalizing these contracts early, the development tracks were successfully decoupled. The mobile application team was able to immediately begin implementing the user interface, utilizing simulated data payloads that perfectly mimicked the agreed-upon structures. Simultaneously, the backend team was able to construct the database schema, implement the security validation logic, and build the application gateway without waiting for the frontend interface to be completed. This parallelization drastically accelerated the overall development timeline and completely eliminated the chaotic integration conflicts that frequently plague large-scale academic engineering projects.

4.2. Artificial Intelligence and Vision Pipeline Implementation

The development of the system's core capability, the automated visual detection of bovine estrus, represented the most mathematically complex and computationally demanding phase of the project. The implementation of this pipeline was divided into three distinct stages: dataset curation, model training, and inference optimization.

4.2.1. Dataset Curation and Model Fine-Tuning

Standard, commercially available object detection models are pre-trained to recognize generic, everyday objects such as vehicles or human pedestrians. They possess zero inherent capability to identify specific agricultural livestock, let alone differentiate between a cow standing idle and a cow actively exhibiting reproductive mounting behavior.

To overcome this, the artificial intelligence sub-team undertook the massive task of curating a highly specialized, proprietary dataset. This involved acquiring hundreds of hours of raw, unedited barn surveillance footage capturing various dairy herds. The team meticulously analyzed this footage frame-by-frame, manually drawing precise geometric bounding boxes around cattle and explicitly categorizing their physical behaviors.

Once this proprietary dataset was constructed, the team utilized it to fundamentally retrain a state-of-the-art neural network architecture through a process known as transfer learning. By adjusting the internal mathematical weights and utilizing specialized loss functions to penalize incorrect behavioral categorizations, the model was incrementally taught to recognize the specific visual signature of bovine estrus with a high degree of statistical confidence.

4.2.2. Real-Time Inference and Algorithmic Deduplication

Implementing the trained model into a live, real-time video stream presented immediate computational challenges. Processing high-definition video at standard frame rates completely saturated the graphical processing units, introducing unacceptable latency that rendered real-time notification impossible.

The engineering team implemented two critical optimizations to resolve this bottleneck. First, a dynamic frame-sampling algorithm was introduced. Recognizing that the physical act of bovine mounting is a relatively slow biological process, the algorithm purposefully discards the vast majority of incoming video frames, extracting and analyzing only a precisely calculated fraction of frames per second. Second, these extracted frames undergo a rapid, in-memory tensor manipulation process that mathematically compresses their resolution to perfectly match the internal dimensions of the neural network, drastically reducing the computational overhead.

Following inference, the system implements the vital spatial deduplication algorithm. As the model outputs multiple positive detections for a single sustained event, the algorithm calculates the geometric intersection over union across consecutive frames. The implementation utilizes a strict similarity threshold; if the spatial overlap between bounding boxes in frame A and frame B exceeds this threshold, the algorithm fuses the data structures in memory, recognizing them as a continuous temporal event. The algorithm only finalizes the record and passes it to the central server when the spatial overlap drops below the threshold, indicating the physical interaction has definitively concluded.

4.3. Central Application Gateway and Backend Implementation

The central application gateway was constructed utilizing a highly performant, asynchronous Python-based web framework. The implementation of this layer focused heavily on execution speed, absolute data integrity, and uncompromising security.

4.3.1. Security Middleware and Identity Verification

The implementation of the security perimeter was treated as the paramount engineering priority. The team developed custom middleware, a layer of protective logic that wraps the entire application. When a request arrives from a mobile client, this middleware intercepts the transmission before it can access any internal database queries.

The middleware extracts the cryptographic authorization token from the transmission headers and utilizes asymmetric cryptographic keys to mathematically verify the token's signature against the cloud identity provider. Once the signature is verified, the middleware extracts the user's unique identifier and actively queries the relational database to verify the user's specific role and their authorized association with the requested farm environment. If any of these mathematical or relational checks fail, the middleware instantaneously terminates the connection, returning a standardized unauthorized response and logging the aborted attempt for administrative review.

4.3.2. The Reproductive State Machine

A critical component of the backend implementation is the automated reproductive state machine. Rather than relying on simple database updates, the team implemented a complex, event-driven logical engine. When a farm worker logs a biological event, such as a confirmed insemination, the state machine intercepts this data. It validates the biological feasibility of the event (for example, ensuring a cow cannot be logged as inseminated if it was logged as giving birth three days prior), updates the animal's primary status, and simultaneously calculates projected future events utilizing standardized veterinary gestation timelines. This ensures the digital herd registry remains mathematically consistent and biologically accurate without requiring manual calculations by the farmer.

4.4. Client Application and User Interface Implementation

The presentation layer was implemented utilizing a modern, declarative framework designed for compiling native applications across both mobile and web platforms from a single, unified codebase. The implementation prioritized fluid user experiences, aggressive local data caching, and graceful error handling.

4.4.1. Reactive State Management Implementation

To manage the immense complexity of asynchronous network requests and dynamic user interfaces, the frontend team implemented a sophisticated reactive state management architecture. In this implementation, the visual interface elements are mathematically bound to specific data providers residing in the device's volatile memory.

When the user requests an action, such as confirming a pending estrus alert, the application dispatches a secure network request to the central server. Concurrently, the state manager places the specific visual element into a localized loading state, providing immediate visual feedback to the user. Once the central server returns a successful cryptographic confirmation, the state manager automatically mutates the local data cache. Because the visual

elements are reactively bound to this cache, they instantly and automatically redraw themselves to reflect the updated state, entirely removing the alert from the pending queue without requiring a manual screen refresh.

4.4.2. Cross-Platform Video Rendering Pipeline

The most significant frontend implementation challenge was rendering the live barn surveillance footage consistently across diverse mobile operating systems and web browsers, as native network streaming protocols lack universal support.

To solve this, the team implemented a custom rendering pipeline. The central server was programmed to decode the raw network stream and transmit it outward as a continuous sequence of compressed, discrete images. On the client device, a highly optimized memory buffer was implemented to receive this byte stream. A custom graphical canvas element was then engineered to continuously paint the incoming bytes onto the screen, aggressively flushing older frames from the device's memory to prevent application crashes. This implementation successfully created the illusion of fluid, high-definition video playback natively within the application, entirely circumventing the need for third-party media player plugins.

4.5. Infrastructure, Containerization, and Deployment

The final phase of implementation involved transitioning the software from isolated local development machines into a secure, highly available production server environment. This was achieved through rigorous infrastructure-as-code and containerization methodologies.

Every major subsystem of the relational database, the application gateway, and the vision processing worker was meticulously packaged into its own isolated, virtualized container. These containers explicitly define their own necessary operating system libraries, software dependencies, and internal environmental variables. This implementation guarantees absolute environmental parity; the exact code that executed successfully on a developer's laptop will execute identically on the production server, completely eliminating unpredictable deployment failures.

To orchestrate these virtualized containers, the team implemented a declarative composition file that defines the internal network topology. The application gateway container and the vision worker container are securely connected to the database container via an internal, unexposed network bridge. To connect the system to the public internet, a highly optimized reverse proxy server was implemented as the single point of entry. This proxy server is responsible for receiving encrypted internet traffic, terminating the secure socket layer connection, and safely routing the decrypted request to the appropriate internal container, ensuring the internal architecture remains entirely shielded from direct external access.

5. Test Cases and Results

To rigorously guarantee that the Moo-tion platform meets the uncompromising demands of a commercial agricultural environment, an exhaustive and highly formalized software testing

strategy was developed and executed. The testing philosophy encompassed multiple layers of software validation, beginning with localized unit tests targeting isolated algorithmic functions, progressing to complex integration testing across the distributed architecture, and culminating in comprehensive end-to-end system testing utilizing simulated farm conditions.

Recognizing the critical nature of reproductive timing in dairy farming, where a missed alert equates to significant financial loss, the testing matrix was designed with a heavy emphasis on system fault tolerance, multi-tenant data security, and artificial intelligence precision. The following section details fifty distinct, documented test cases divided across seven core operational domains. All test cases outlined below were successfully executed and verified prior to the final system deployment.

5.1 Functional Test Cases

5.1.1 User Account and Authentication Test Cases

Test ID	FR-AUTH-001A	Category	Security
Objective	To verify that a user can successfully log in with valid credentials.		
Steps	1. Open the mobile application. 2. Navigate to the Login page 3. Enter a valid email address and password 4. Click the "Login" button.		
Expected	The system should authenticate the user and allow access to the dashboard.		
Tested By	Canberk Özba		
Date-Result	15.11.2025 / Success		

Test ID	FR-AUTH-001B	Category	Security
Objective	To verify that the system rejects login attempts with invalid credentials.		
Steps	1. Open the mobile application 2. Navigate to the Login page. 3. Enter an invalid email or password. 4. Click the "Login" button.		
Expected	The system should deny access and display an authentication error message.		
Tested By	Mert Özkaya		
Date-Result	15.11.2025 / Success		

Test ID	FR-AUTH-002	Category	Security
Objective	To verify that protected system resources require authentication.		
Steps	1. Open the application. 2. Attempt to access the dashboard without logging in.		
Expected	The system should redirect the user to the login page.		

Tested By	Muhammed Enes Şahin
Date-Result	15.11.2025 / Success

Test ID	FR-AUTH-003	Category	Security
Objective	To verify that role-based access control restricts unauthorized actions.		
Steps	1. Login as a Farm Worker. 2. Attempt to modify the camera settings.		
Expected	The Farm Workers should not be allowed to change the settings.		
Tested By	Emir Eroğlu		
Date-Result	09.01.2026 / Success		

Test ID	FR-AUTH-004	Category	Security
Objective	To verify that role-based access control restricts unauthorized actions.		
Steps	1. Login as a Farm Manager.		

	2. Attempt to modify the camera settings.
Expected	The system successfully changed the camera settings.
Tested By	Ahmet Burak Kılıçkaya
Date-Result	12.01.2026 / Success

Test ID	FR-AUTH-005	Category	Security
Objective	To verify that the system invalidates the user session after logout.		
Steps	1. Log in successfully to the mobile application. 2. Navigate to the settings or account menu. 3. Click the "Logout" button. 4. Attempt to access the dashboard using the previous session.		
Expected	The system should terminate the session and redirect the user to the login page.		
Tested By	Mert Özkaya		
Date-Result	15.01.2026 / Success		

Test ID	FR-AUTH-006	Category	Security
Objective	To verify that a Farm Worker cannot create, edit, or delete user accounts.		
Steps	1. Log in as a Farm Worker. 2. Navigate to any available user or account management interface. 3. Attempt to create a new user. 4. Attempt to edit an existing user. 5. Attempt to delete an existing user.		
Expected	The system should deny all account management actions for the Farm Worker role.		
Tested By	Canberk Özba		
Date-Result	17.01.2026 / Success		

Test ID	FR-AUTH-007	Category	Security
Objective	To verify that the system rejects API requests made with an expired authentication token.		
Steps	1. Log in to obtain a valid authentication token. 2. Wait until the token expires or manually simulate expiration.		

	3. Send a request to a protected API endpoint using the expired token.
Expected	The system should reject the request and return an authentication error.
Tested By	Emir Eroğlu
Date-Result	20.01.2026 / Success

Test ID	FR-AUTH-008	Category	Security
Objective	To verify that a new user can successfully create an account using the signup process.		
Steps	1. Open the mobile application. 2. Navigate to the Signup page. 3. Enter valid name, email, and password. 4. Click the "Sign Up" button.		
Expected	The system should create a new user account and redirect the user to the email verification page.		
Tested By	Muhammed Enes Şahin		
Date-Result	15.11.2025 / Success		

Test ID	FR-AUTH-009	Category	Security
Objective	To verify that email verification succeeds with a valid verification code.		
Steps	1. Complete the signup process. 2. Receive the verification code by email. 3. Enter the correct verification code in the verification screen. 4. Submit the code.		
Expected	The system should verify the user account and allow access to the farm setup or dashboard.		
Tested By	Mert Özkaya		
Date-Result	15.11.2025 / Success		

Test ID	FR-AUTH-010	Category	Security
Objective	To verify that the system rejects incorrect email verification codes.		
Steps	1. Complete the signup process. 2. Enter an incorrect verification code in the verification screen. 3. Submit the code.		

Expected	The system should display an error message and prevent account activation.
Tested By	Canberk Özba
Date-Result	15.11.2025 / Success

5.1.2 Farm and User Management Test Cases

Test ID	FR-FARM-001	Category	Functional
Objective	To verify that a user can create a farm profile after account registration.		
Steps	1. Log in to the application. 2. Navigate to the farm creation page. 3. Enter farm name, location, and herd size. 4. Save the configuration.		
Expected	The system should create the farm profile and associate it with the user account.		
Tested By	Emir Eroğlu		
Date-Result	15.11.2025 / Success		

Test ID	FR-FARM-002	Category	Functional
Objective	To verify that a Farm Owner can invite a new staff member to the farm system..		
Steps	1. Log in as Farm Owner. 2. Navigate to the Manage Users page. 3. Click "Invite User". 4. Enter the staff member’s email and assign a role. 5. Send the invitation.		
Expected	The system should send an invitation email and create a pending user account.		
Tested By	Muhammed Enes Şahin		
Date-Result	15.11.2025 / Success		

Test ID	FR-FARM-003	Category	Functional
Objective	To verify that invited staff members can activate their account through an invitation link.		
Steps	1. Receive invitation email. 2. Click the invitation link.		

	3. Set a password. 4. Log in to the system.
Expected	The system should activate the user account with the assigned role permissions.
Tested By	Emir Eroğlu
Date-Result	15.11.2025 / Success

5.1.3 Camera Management Test Cases

Test ID	FR-CAM-001	Category	Functional
Objective	To verify that the system successfully registers a new camera configuration.		
Steps	1. Farm Manager Login to the application. 2. The user adds a new camera configuration.		
Expected	The system should register the camera and display it in the camera list.		
Tested By	Ahmet Burak Kılıçkaya		
Date-Result	15.11.2025 / Success		

Test ID	FR-CAM-002	Category	Functional
Objective	To verify that the system handles invalid camera stream configurations.		
Steps	1. Log in as a Farm Manager. 2. Navigate to the camera configuration page. 3. Enter an invalid RTSP URL. 4. Attempt to save the configuration.		
Expected	The system should reject the configuration and display an error message.		
Tested By	Mert Özkaya		
Date-Result	13.02.2026 / Success		

Test ID	FR-CAM-003	Category	Functional
Objective	To verify that a Farm Manager can edit an existing camera configuration.		
Steps	1. Log in as a Farm Manager. 2. Navigate to the Camera Management page. 3. Select an existing camera from the list.		

	4. Modify the camera location or RTSP URL. 5. Save the updated configuration.
Expected	The system should update the camera configuration and reflect the changes in the camera list.
Tested By	Muhammed Enes Şahin
Date-Result	17.02.2026 / Success

Test ID	FR-CAM-004	Category	Functional
Objective	To verify that a Farm Manager can delete a camera configuration.		
Steps	1. Log in as a Farm Manager. 2. Navigate to the Camera Management page. 3. Select an existing camera. 4. Click the "Delete" option. 5. Confirm the deletion.		
Expected	The system should remove the camera from the camera list and stop processing its stream.		
Tested By	Ahmet Burak Kılıçkaya		

Date-Result	21.02.2026 / Success

Test ID	FR-Cam-005	Category	Functional
Objective	To verify that the system successfully ingests RTSP camera streams.		
Steps	1. Configure a valid RTSP camera. 2. Start the vision pipeline. 3. Monitor the incoming video stream.		
Expected	The system should successfully receive and process frames from the RTSP stream.		
Tested By	Emir Eroğlu		
Date-Result	26.02.2026 / Success		

5.1.4 Vision Pipeline and Detection Test Cases

Test ID	FR-AI-001	Category	Functional
Objective	To verify that the system detects cattle in video frames.		
Steps	1. Start the vision pipeline.		

	2. Provide a camera feed containing cattle. 3. Observe the detection results.
Expected	The system should detect cattle in the frame using the object detection model.
Tested By	Muhammed Enes Şahin
Date-Result	03.03.2026 / Success

Test ID	FR-AI-002	Category	Functional
Objective	To verify that the system tracks detected cows across frames.		
Steps	1. Start the camera stream. 2. Detect a cow in the first frame. 3. Continue processing subsequent frames.		
Expected	The system should continuously track the same cow across frames.		
Tested By	Mert Özkaya		
Date-Result	10.03.2026 / Partially successful. The system can process detections across frames, but continuous identity tracking for the same cow is not fully validated. In crowded scenes or after movement between frames, the same cow may not always be preserved as a stable tracked target.		

Test ID	FR-AI-003	Category	Functional
Objective	To verify that the system detects estrus mounting behavior.		
Steps	1. Provide a video stream containing mounting behavior. 2. Run the behavior analysis module. 3. Observe the detection results.		
Expected	The system should detect mounting behavior and classify it as an estrus event.		
Tested By	Ahmet Burak Kılıçkaya		
Date-Result	08.03.2026 / Success		

Test ID	FR-AI-004	Category	Edge Case
Objective	To verify that incomplete mounting behavior does not trigger an estrus alert.		
Steps	1. Provide a video where mounting behavior lasts less than the detection threshold. 2. Run the behavior detection pipeline.		
Expected	The system should not generate an estrus alert.		

Tested By	Muhammed Enes Şahin
Date-Result	12.03.2026 / Partially successful. The system can avoid alerts when the model does not produce a confident detection, but the duration-based incomplete mounting threshold has not been fully validated. Very short mounting-like behavior may still require additional tuning to prevent false alerts.

5.1.5 Notification and Alert Test Cases

Test ID	FR-NOTIF-001	Category	Integration
Objective	To verify that the system sends push notifications when an estrus event is detected.		
Steps	1. Trigger an estrus detection event. 2. Allow the backend to process the event. 3. Observe the registered mobile device.		
Expected	The system should send a push notification to the user device.		
Tested By	Emir Eroğlu		
Date-Result	16.03.2026 / Success		

5.1.6 Integration Test Cases

Test ID	FR-INT-001	Category	Integration

Objective	To verify that a detected mounting event is processed end-to-end from the edge vision pipeline to the mobile application alert screen.
Steps	1. Configure a valid RTSP camera for a farm. 2. Start the vision pipeline connected to the configured camera. 3. Provide a video stream containing a valid mounting behavior exceeding the detection threshold. 4. Allow the detection module to identify the target event. 5. Allow the tracking and behavior evaluation modules to confirm the event. 6. Send the event payload to the backend API. 7. Allow the backend to store the event metadata and trigger a push notification. 8. Open the mobile application with a user account assigned to the same farm. 9. Navigate to the alert history screen.
Expected	The system should detect the event, store its metadata, send a notification, and display the alert in the mobile application for the correct farm user.
Tested By	Canberk Özba
Date-Result	24.03.2026 / Success

Test ID	FR-INT-002	Category	Integration

Objective	To verify that the system stores an event snapshot in cloud storage and links it correctly to the event record in the database.
Steps	1. Start the vision pipeline for a configured farm camera. 2. Trigger a valid estrus-related event. 3. Allow the pipeline to generate a cropped JPEG snapshot. 4. Upload the snapshot to the configured cloud storage service. 5. Store the returned snapshot URL in the event log record. 6. Query the event record from the backend. 7. Open the same event from the mobile application.
Expected	The system should upload the snapshot successfully, save the snapshot URL in the event record, and display the same image when the event is opened in the mobile application.
Tested By	Ahmet Burak Kılıçkaya
Date-Result	19.03.2026 / Success

Test ID	FR-INT-003	Category	Integration
Objective	To verify that an authenticated mobile client can retrieve alert history together with the corresponding media references from the backend.		
Steps	1. Log in the mobile application using valid credentials. 2. Ensure that at least one event with a snapshot exists in the system.		

	3. Request alert history from the backend. 4. Select one alert entry from the list. 5. Request the associated snapshot.
Expected	The system should return the alert history for the authorized farm and provide access to the associated media reference for each alert that includes a snapshot.
Tested By	Mert Özkaya
Date-Result	21.03.2026 / Success

Test ID	FR-INT-004	Category	Integration
Objective	To verify that the tracking subsystem maintains the target identity during short-term occlusion.		
Steps	1. Start the vision pipeline on a stream containing multiple cows. 2. Detect a target cow exhibiting relevant behavior. 3. Introduce a short-term occlusion where the target is partially hidden by another cow or a barn obstacle. 4. Continue processing subsequent frames.		
Expected	The system should continue tracking the same target after the brief occlusion without assigning a new identity unnecessarily.		

Tested By	Muhammed Enes Şahin
Date-Result	28.03.2026 / Partially successful. The system can continue processing after short visual interruptions, but true identity preservation during occlusion is not fully implemented. If a cow is briefly hidden by another cow or a barn object, the target may not always retain the same identity.

Test ID	FR-INT-005	Category	Integration
Objective	To verify that the backend logs detected events into the database with the correct metadata.		
Steps	1. Submit a valid event payload to the backend. 2. Query the event logs in the database. 3. Compare the stored record with the submitted payload		
Expected	The backend should store the event timestamp, camera ID, behavior classification, farm association, and snapshot reference correctly.		
Tested By	Emir Eroğlu		
Date-Result	18.03.2026 / Success		

5.2 Non Functional Test Cases

5.2.1 Performance Test Cases

Test ID	NFR-PERF-001	Category	Performance
Objective	To verify that critical estrus alerts are delivered to the mobile application within the required time threshold (5 seconds).		
Steps	Start the RTSP camera stream, vision pipeline, backend service, and mobile application. Provide a valid mounting behavior event that exceeds the detection threshold. Record the time when the threshold is crossed in the vision pipeline. Record the time when the push notification is received on the mobile device.		
Expected	The system should deliver the alert to the mobile application within 5 seconds after the behavioral threshold is crossed.		
Tested By	Ahmet Burak Kılıçkaya		
Date-Result	03.04.2026 / Success		

Test ID	NFR-PERF-002	Category	Performance
Objective	To verify backend API response time under normal load.		

Steps	1. Start the backend server. 2. Send requests to common endpoints (login, alert history, camera list). 3. Measure response time.
Expected	The backend should respond to requests within 500 ms under normal system load.
Tested By	Ahmet Burak Kılıçkaya
Date-Result	29.03.2026 / Success

Test ID	NFR-PERF-003	Category	Performance
Objective	To verify real-time processing of camera streams.		
Steps	1. Start the RTSP camera stream. 2. Run the vision pipeline continuously. 3. Measure processing delay between frame capture and inference output.		
Expected	The vision pipeline should process frames with maximum latency of 1 second.		
Tested By	Mert Özkaya		

Date-Result	01.04.2026 / Success

Test ID	NFR-PERF-004	Category	Performance
Objective	To verify event snapshot retrieval performance.		
Steps	1. Ensure event snapshots are stored in cloud storage. 2. Open an alert entry in the mobile application. 3. Measure the time required for the snapshot image to load.		
Expected	The snapshot image should load within 1.5 seconds on a stable network connection.		
Tested By	Muhammed Enes Şahin		
Date-Result	06.04.2026 / Success		

5.2.2 Usability Test Cases

Test ID	NFR-USE-001	Category	Usability
Objective	To verify dashboard readability.		
Steps	1. Log in to the mobile application. 2. Open the monitoring dashboard.		

Expected	Users should be able to locate alert information within 5 seconds without external guidance.
Tested By	Emir Eroğlu
Date-Result	14.03.2026 / Success

Test ID	NFR-USE-002	Category	Usability
Objective	To verify that alerts can be easily interpreted.		
Steps	1. Generate multiple alerts in the system. 2. View the alert history page.		
Expected	Users should correctly interpret alert details such as camera location and event time within 5 seconds.		
Tested By	Ahmet Burak Kılıçkaya		
Date-Result	17.03.2026 / Success		

5.2.3 Reliability Test Cases

Test ID	NFR-REL-001	Category	Reliability

Objective	To verify continuous system operation.
Steps	1. Run the full system pipeline (camera, vision, backend). 2. Operate the system continuously.
Expected	The system should operate continuously for at least 72 hours without critical service failure.
Tested By	Canberk Özba
Date-Result	30.04.2026 / Success

Test ID	NFR-REL-002	Category	Reliability
Objective	To verify system recovery from camera interruption.		
Steps	1. Start the system with a valid RTSP stream. 2. Disconnect the camera network for 30 seconds. 3. Restore the connection.		
Expected	The system should resume normal stream processing within 10 seconds after reconnection.		
Tested By	Muhammed Enes Şahin		

Date-Result	09.04.2026 / Success

Test ID	NFR-REL-003	Category	Reliability
Objective	To verify that stored data remains consistent after an unexpected shutdown or power outage.		
Steps	1. Start the full system and process active detections and alerts. 2. During operation, simulate an unexpected shutdown or power loss. 3. Restart the backend, database, and vision services. 4. Query the latest stored event, alert, and tracking records.		
Expected	The system should restart successfully, and previously stored data should remain consistent without corruption, duplication, or missing committed records.		
Tested By	Mert Özkaya		
Date-Result	18.04.2026 / Success		

Test ID	NFR-REL-004	Category	Reliability
Objective	To verify that video processing tolerates temporary network drops or camera signal loss without losing long-term tracking continuity.		

Steps	1. Start a valid camera stream and tracking pipeline. 2. Allow the system to track one or more cows over time. 3. Introduce a short network interruption or temporary signal loss. 4. Restore the connection and continue processing.
Expected	The system should recover after the temporary interruption and continue operation without losing long-term tracking data unnecessarily.
Tested By	Canberk Özba
Date-Result	12.04.2026 / Success

5.2.4 Scalability Test Cases

Test ID	NFR-SCAL-001	Category	Scalability
Objective	To verify support for multiple camera streams.		
Steps	1. Configure multiple RTSP cameras. 2. Run the vision pipeline.		
Expected	The system should support at least 5 simultaneous camera streams without critical performance degradation.		

Tested By	Mert Özkaya
Date-Result	15.04.2026 / Success.

Test ID	NFR-SCAL-002	Category	Scalability
Objective	To verify system performance under increased event volume.		
Steps	1. Simulate or submit multiple event detections rapidly. 2. Monitor backend processing and storage.		
Expected	The backend should process at least 100 event submissions per minute without data loss.		
Tested By	Canberk Özba		
Date-Result	22.04.2026 / Success		

Test ID	NFR-SCAL-003	Category	Scalability
Objective	To verify database performance with large event history.		
Steps	1. Populate the database with 10,000 event records.		

	2. Retrieve alert history through the mobile application.
Expected	The system should retrieve and display event records within 3 seconds.
Tested By	Muhammed Enes Şahin
Date-Result	20.04.2026 / Success

Test ID	NFR-SCAL-004	Category	Scalability
Objective	To verify that the system scales to farms with at least 50 cows and 4 simultaneous camera streams.		
Steps	1. Configure the system for a farm scenario containing at least 50 cows. 2. Connect 4 simultaneous camera streams. 3. Run the vision pipeline, backend processing, and mobile alert flow under this load. 4. Monitor throughput, responsiveness, and stability.		
Expected	The system should continue operating correctly at the target scale without critical failures or unacceptable degradation in core functions.		
Tested By	Canberk Özba		

Date-Result	25.04.2026 / Success

5.2.5 Compatibility Test Cases

Test ID	NFR-COMP-001	Category	Compatibility
Objective	To verify that the mobile application operates correctly on supported Android devices.		
Steps	1. Install the mobile application on a supported Android device. 2. Log in using valid credentials. 3. Open the dashboard, alert history, and alert detail pages. 4. Open an event snapshot image.		
Expected	The system should function correctly on supported Android devices without major interface or functional inconsistencies.		
Tested By	Ahmet Burak Kılıçkaya		
Date-Result	27.03.2026 / Success		

Test ID	NFR-COMP-002	Category	Compatibility

Objective	To verify that the mobile application operates correctly on supported iOS devices.
Steps	1. Install the mobile application on a supported iOS device. 2. Log in using valid credentials. 3. Open the dashboard, alert history, and alert detail pages. 4. Open an event snapshot image.
Tested By	Mert Özkaya
Expected	The system should function correctly on supported iOS devices without major interface or functional inconsistencies.
Tested By	Emir Eroğlu
Date-Result	04.04.2026 / Success

6. Maintenance Plan and Details

To guarantee that the Moo-tion platform remains a dependable, accurate, and commercially viable decision-support tool over years of continuous agricultural operation, the engineering team has formulated a rigorous, multi-tiered maintenance strategy. This plan directly addresses the inevitable realities of software entropy, physical hardware degradation in harsh barn environments, and the natural degradation of artificial intelligence accuracy.

The maintenance architecture is divided into three distinct operational categories: Infrastructure and Relational Data Maintenance, Artificial Intelligence and Active Learning Maintenance, and Hardware Ecosystem Support.

6.1. Infrastructure and Relational Data Maintenance

The underlying software infrastructure, comprising the central application gateway and the relational database, requires disciplined lifecycle management to prevent data corruption and ensure uninterrupted service availability.

6.1.1. Automated Database Schema Migrations

As the Moo-tion platform evolves, perhaps integrating new features such as lameness detection or automated feeding tracking, the structural schema of the relational database must inevitably change. To execute these modifications safely within a live production environment, all database mutations will be managed exclusively via automated, version-controlled migration scripts (e.g., utilizing Alembic for SQLAlchemy).

This approach ensures that schema changes are additive and rigorously tested in staging environments before production deployment. Crucially, it provides a mathematical guarantee that if a schema update introduces a critical error, the infrastructure can automatically roll back the database to its previous stable state without any loss of historical animal data.

6.1.2. Cloud Storage Retention and Archival Policies

The system generates thousands of high-resolution, annotated visual evidence frames monthly. While this evidence is critical for immediate human verification, maintaining years of high-resolution image data in "hot" cloud storage incurs significant, compounding operational costs.

To mitigate this, an automated data lifecycle policy has been implemented. Background chronometer tasks will execute daily, scanning the relational database for alerts that have exceeded a predefined historical relevance window (e.g., 180 days). The associated image files in the cloud object storage bucket will be permanently purged, immediately reducing overhead costs. Importantly, the lightweight textual metadata that records the event occurred, when it occurred, and which animal was involved, will remain permanently preserved within the relational database for long-term veterinary analytics.

6.2. Artificial Intelligence and Active Learning Maintenance

A computer vision model trained on a static dataset will eventually experience a degradation in accuracy. Agricultural environments are highly dynamic: seasonal variations alter natural barn lighting, facility renovations change background architecture, and farmers may introduce entirely new cattle breeds with different physical dimensions. This phenomenon, known as concept drift, necessitates continuous model maintenance.

6.2.1. The Active Learning Feedback Loop

To proactively combat concept drift, the engineering team engineered a continuous "Active Learning Loop" directly into the user interface. When a farm worker reviews an alert and explicitly categorizes it as a "False Positive" or "Incorrect Animal" within the application, the system does not simply discard the event. Instead, it archives that specific visual evidence

frame, alongside its algorithmic metadata and the user's explicit correction, into a dedicated, secure training repository.

6.2.2. Periodic Model Fine-Tuning and Re-Deployment

On a biannual maintenance schedule, the engineering team will aggregate this repository of difficult edge cases and historical algorithmic failures. This highly valuable, human-verified dataset will be utilized to retrain and fine-tune the core YOLO-based object detection model.

By continuously feeding the model examples of its own mistakes, the artificial intelligence incrementally learns to navigate complex lighting scenarios, ignore confusing background architecture, and improve its intersection-over-union deduplication accuracy. Following retraining and rigorous benchmark testing, the updated model weights will be containerized and silently deployed to the production worker instances, compounding the system's precision and reliability over its operational lifetime without requiring any active input from the farmer.

6.3. Hardware Ecosystem and Network Support

While Moo-tion is a software platform, its efficacy is inextricably linked to the physical health of the farmer's surveillance infrastructure. The harsh reality of a dairy barn characterized by high humidity, excessive dust, ammonia vapors, and physical vibration means that cameras will inevitably fail or become obscured.

6.3.1. Automated Camera Health Polling

The vision processing worker is programmed to conduct continuous, background health polling on all registered network streams. If a stream unexpectedly terminates or suffers severe packet loss, the system initiates an exponential backoff reconnection sequence. If the camera remains unresponsive after a predefined interval, the system automatically generates an administrative alert, notifying the farm manager via push notification that a specific hardware zone (e.g., "Maternity Pen Camera 2") requires physical inspection.

6.3.2. Visual Degradation Monitoring

Beyond total hardware failure, the system must account for progressive visual obstruction, such as dust accumulating on the camera lens or a spider web obscuring the field of view. As part of future maintenance expansions, the vision engine will include a baseline image-quality heuristic. If the system detects a sustained, severe drop in overall image contrast or sharpness across a prolonged period, indicating a dirty lens rather than simply nighttime conditions, it will preemptively suggest hardware cleaning to the facility manager, ensuring the artificial intelligence always receives the highest quality visual data possible.

7. Other Project Elements

The engineering of a commercial-scale agricultural platform extends significantly beyond the pure implementation of software logic. A successful senior design project must rigorously evaluate the broader context in which the technology operates. This section details the external constraints that shaped the architectural design, the international standards adhered to, the ethical frameworks guiding the development process, and the collaborative team dynamics that ensured the project's ultimate success.

7.1. Consideration of Various Factors in Engineering Design

The translation of theoretical computer science concepts into a functional agricultural tool requires the careful navigation of numerous real-world limiting factors. The design of the Moo-tion platform was heavily influenced by computational, environmental, regulatory, economic, and social constraints.

7.1.1. Engineering and Environmental Constraints

- **Computational and Hardware Limitations:** Real-time artificial intelligence inference, specifically object detection applied to high-definition video feeds, demands immense graphical processing capabilities. The primary constraint encountered during the design phase was balancing this computational demand against the economic realities of agricultural hardware. To mitigate this absolute constraint, the engineering team designed the vision processing pipeline to operate efficiently on mid-tier consumer hardware through aggressive algorithmic optimization, specifically the implementation of frame-sampling techniques and dynamic resolution downscaling.
- **Environmental and Physical Realities:** The physical environment of a commercial dairy barn presents severe constraints. Barns are characterized by high humidity, fluctuating natural lighting, airborne particulate matter, and corrosive ammonia vapors, which directly degrade the optical clarity of standard surveillance hardware. Consequently, the model training required the purposeful injection of visual noise, teaching the neural network to identify geometric bovine structures even when the camera lens is partially obscured or operating in low-light, high-contrast scenarios.
- **Rural Network Latency:** Dairy farms are frequently situated in geographically isolated rural areas characterized by notoriously unstable internet connectivity. This constraint dictated the distributed architecture, forcing the heavy video processing to occur locally or on highly optimized edge instances, while restricting outgoing network traffic exclusively to lightweight, compressed notification images and purely textual metadata payloads.

7.1.2. Economic and Social Factors

- **Economic Constraints:** Learning and adoption should be affordable for agricultural operations of all sizes. Economic factors influenced the choice of open-source frameworks, cost-efficient cloud storage solutions, and the decision to utilize the farmer's

preexisting network cameras rather than forcing the purchase of proprietary hardware. This ensures the platform remains financially accessible while still allowing for sustainable monetization through tiered farm-management features.

- **Social Impact:** Social inclusivity and worker wellbeing are primary considerations. Continuous manual observation of cattle during late-night hours places a severe physical and mental burden on farm laborers. By automating this process, the system reduces nighttime labor requirements, directly contributing to improved workplace conditions and the social welfare of the agricultural workforce.

7.1.3. Interoperability and Regulatory Standards

- **Network and Streaming Protocols:** To prevent vendor lock-in, the video ingestion pipeline was explicitly designed to adhere to the standard Real-Time Streaming Protocol. This guarantees plug-and-play interoperability with the vast majority of commercial surveillance hardware available globally.
- **Data Privacy and Regulatory Compliance:** The architectural foundation was profoundly influenced by the necessity to adhere to modern data protection legislation, specifically the Turkish Personal Data Protection Law (KVKK) and the General Data Protection Regulation (GDPR). The system architecture complies with these standards by enforcing absolute cryptographic data isolation between different farm environments and ensuring that visual evidence is securely encrypted.

7.2. Ethics and Professional Responsibilities

The conceptualization, development, and deployment of the Moo-tion platform were rigorously guided by established codes of ethics and professional conduct inherent to the computer science and engineering disciplines.

- **Prioritizing Animal Welfare:** The engineering team recognized an absolute ethical mandate to ensure the software actively supports animal well-being. The platform is explicitly designed as a human decision-support tool. It does not possess the capability to execute automated veterinary decisions without human oversight. The system operates solely by notifying the human caretaker, ensuring that an experienced "human-in-the-loop" visually verifies the biological condition before any physical interventions are undertaken. Furthermore, the completely non-invasive nature of the software acts as an ethical alternative to physical biometric tracking collars.
- **Safeguarding Worker Privacy and Data Minimization:** The team explicitly acknowledged the severe ethical dilemma inherent in deploying continuous surveillance cameras within a shared human workplace. To fulfill our professional responsibilities regarding employee privacy, strict algorithmic data minimization practices were structurally implemented into the core vision engine. The system is computationally programmed to permanently save only the specific, temporally isolated video frames containing a confirmed, biologically relevant animal detection. All surrounding video data capturing general human movement is immediately, automatically, and irreversibly discarded from the system's volatile memory.

7.3. Teamwork Details

The complexity of engineering a distributed, multi-platform agricultural ecosystem required a highly disciplined approach to team dynamics. To ensure efficiency and inclusivity, the team followed a structured division of work, utilizing agile methodologies and maintaining regular communication through daily updates and peer coding practices.

7.3.1. Contributing and Functioning Effectively

Each team member played a critical role in different aspects of the application's design, development, and implementation. Tasks were distributed based on the unique skill sets and workloads of the team members.

- **Ahmet Burak Kılıçkaya**
  - Led the mobile application development, establishing the cross-platform codebase.
  - Designed the reactive state management architecture, ensuring the mobile interface updated instantaneously without frame drops.
  - Implemented the domain localization logic, ensuring the application natively supported both Turkish and English agricultural terminology.
  - Designed the user authentication screens, onboarding workflows, and the primary alert dashboard interface.
  - Authored comprehensive unit tests for the presentation layer to guarantee interface stability.
- **Canberk Özba**
  - Led the core backend architecture and system orchestration.
  - Designed and implemented the relational database schemas, establishing the strict foreign key constraints required for multi-tenant data isolation.
  - Developed the "reproductive state machine," the algorithmic logic that automatically calculates an animal's biological timeline based on historical veterinary inputs.
  - Implemented the role-based access control middleware, ensuring secure privilege escalation blocks.
  - Managed the secure integration between the backend server and the cloud identity provider.
- **Emir Eroğlu**
  - Co-led the presentation subsystem, focusing heavily on the web-based administrative dashboard and overall user experience.
  - Engineered the custom video transcoding rendering pipeline, allowing continuous motion-JPEG streams to be viewed natively within the application without third-party plugins.
  - Conducted usability analysis, ensuring high-contrast visuals and accessible navigation for users operating in harsh outdoor environments.
  - Implemented the interactive animal health rosters and the hardware configuration menus.

    - Contributed extensively to the project documentation, ensuring the user manual aligned with the final software iteration.
- **Mert Özkaya**
    - Led the research and development of the computer vision and artificial intelligence pipeline.
    - Curated, annotated, and managed the proprietary dataset of bovine behaviors required to train the neural network.
    - Optimized the object detection model, implementing the mathematical frame-sampling and resolution downscaling techniques to ensure real-time performance.
    - Designed and programmed the core spatial deduplication algorithm (Intersection over Union) to filter out redundant detections and prevent alert fatigue.
    - Documented the inference benchmarks and model accuracy metrics for the final reports.
- **Muhammed Enes Şahin**
    - Led the cloud infrastructure, DevOps pipelines, and network ingestion systems.
    - Developed the hardware connection manager responsible for securely ingesting and decoding standard network camera streams.
    - Engineered the lease-based distributed locking mechanism, preventing multiple worker processes from colliding when attempting to process the same camera feed.
    - Managed the cloud object storage integration, establishing the secure upload and retrieval pipelines for high-resolution visual evidence frames.
    - Containerized the entire backend and worker ecosystem, ensuring perfect environmental parity between development and production servers.

7.3.2. Helping Create a Collaborative and Inclusive Environment

From the early stages of the project, the team established a collaborative and inclusive work culture, ensuring open communication, mutual support, and shared decision-making.

- **Agile Synchronization:** The team instituted mandatory, bi-weekly agile synchronization meetings. During these sessions, profound technical challenges were analyzed openly, utilizing a collaborative problem-solving approach rather than assigning individual blame for implementation failures.
- **Knowledge Sharing:** Cross-disciplinary knowledge sharing was heavily prioritized. For instance, the members focused on the backend infrastructure and conducted brief technical seminars regarding containerization technologies for the frontend members, ensuring that every individual possessed the capability to independently compile, execute, and debug the complete technology stack locally.
- **Peer Programming:** The team utilized peer programming sessions to tackle the most complex integration points, particularly the hand-off between the artificial intelligence worker's output and the backend gateway's database insertion logic.

7.3.3. Taking Lead Role and Sharing Leadership

Leadership within the engineering group was inherently fluid and deeply domain-specific, purposefully avoiding a rigid, top-down hierarchical structure.

- **Frontend and UX Leads (Ahmet & Emir):** Took the lead during all user interface, accessibility, and client-side architecture reviews. They dictated how agricultural data should be structured and presented to the end-user, guiding the backend team on what data structures were required to build an intuitive dashboard.
- **Backend and Infrastructure Leads (Canberk & Muhammed):** Assumed the primary leadership roles during the architectural design phase of the security flow and structural database schemas. They confidently guided the team's overarching infrastructural decisions, establishing the strict data validation protocols that the rest of the system had to follow.
- **Artificial Intelligence Lead (Mert):** Assumed immediate direction during the critical integration of the computer vision pipelines. He explicitly dictated how raw visual data must be structured, processed, and evaluated, guiding the infrastructure team on the necessary computational resources required to deploy the model effectively.

7.3.4. Meeting the Project Objectives

Referring directly to the initial project planning documentation and early feasibility reports formulated at the project's inception, the team successfully met, and in several areas exceeded, all primary engineering milestones.

The team successfully delivered a fully functioning, production-ready vertical slice of the Moo-tion technology. The secure, multi-tenant architecture was proven entirely robust under stress testing, completely preventing unauthorized data access. The isolated vision worker successfully demonstrated the computational capability to ingest live network video streams and accurately deduplicate complex visual events in real-time without crashing. Finally, the cross-platform client application compiled flawlessly, operating seamlessly across a diverse array of mobile and desktop device ecosystems, successfully bridging the gap between raw hardware observation and actionable agricultural insights.

7.4. New Knowledge Acquired and Applied

Engineering a production-ready software bridge connecting raw, unpredictable video surveillance hardware to highly responsive mobile interfaces demanded the rapid acquisition and immediate application of significant technical skills residing far outside the boundaries of the standard undergraduate computer science curriculum.

- **Reactive Client Architecture:** The frontend engineering team was required to rapidly absorb and apply advanced declarative state management paradigms. Moving away from traditional imperative user interface design, the team learned to construct reactive

data pipelines, handling highly complex, asynchronous network state changes without degrading the frame rate or responsiveness of the application.

- **Distributed Systems and Concurrency:** Crucially, the entire engineering team studied and successfully applied advanced distributed software architectural patterns. The team learned how to properly decouple computationally heavy, long-running worker processes from extremely fast, synchronous data gateways. This required the acquisition of new knowledge regarding distributed database locking mechanisms and lease-based concurrency control, foundational, highly complex concepts vital for modern enterprise-scale software engineering.
- **Applied Artificial Intelligence:** The artificial intelligence sub-team acquired deep, highly applied knowledge in the deployment of real-time object detection models into high-concurrency production environments. Moving beyond theoretical machine learning exercises, the team mastered the programmatic manipulation of raw video frames, dynamic resolution downscaling, tensor mathematics optimization, and low-level memory buffer management to ensure the artificial intelligence could operate within the strict hardware constraints of real-world deployment.

8. Conclusion and Future Work

8.1. Project Conclusion

Moo-tion represents a comprehensive, highly successful engineering undertaking that directly addresses a critical, economically significant bottleneck within the global agricultural sector. The fundamental challenge of modern dairy farming lies in the precise biological timing of the reproductive cycle, where the failure to accurately detect bovine estrus results in compounding financial losses, extended non-productive feeding periods, and diminished overall herd efficiency. By intentionally shifting the operational paradigm away from expensive, invasive, and failure-prone physical biometric sensors, the Moo-tion platform provides a highly scalable, economically accessible alternative rooted entirely in advanced software engineering.

Throughout the development lifecycle, the engineering team successfully bridged the substantial gap between theoretical computer science concepts and practical, harsh agricultural realities. The platform effectively leverages preexisting, standard barn surveillance infrastructure, transforming passive video feeds into a proactive, intelligent decision-support system. The successful synthesis of a highly responsive, cross-platform mobile application, a secure, multi-tenant central application gateway, and an isolated, high-performance artificial intelligence processing engine unequivocally proves the complete technical and commercial viability of this approach.

The system does not merely detect a biological event; it processes, deduplicates, and contextualizes that event, securely delivering actionable visual evidence directly to the facility personnel. By automating the arduous task of continuous nighttime observation, the platform democratizes access to precision agriculture, allowing farms of all scales to optimize their reproductive efficiency while simultaneously prioritizing animal welfare through a completely non-invasive methodology.

8.2. Summary of Key Achievements

The culmination of this project yielded several significant technical achievements that resolved complex engineering problems:

- **Algorithmic Event Deduplication:** The development of the spatial overlap analysis algorithm successfully mitigated the inherent challenge of alert fatigue. By mathematically calculating the intersection over union of geometric bounding boxes across consecutive temporal frames, the system intelligently merges hundreds of positive neural network inferences into a single, cohesive human alert, ensuring the platform remains a valuable tool rather than a disruptive nuisance.
- **Universal Cross-Platform Video Ingestion:** Overcoming the severe limitations of native network streaming protocols across diverse web browsers and mobile operating systems, the team successfully engineered a custom transcoding pipeline. By converting raw network streams into continuous sequences of optimized imagery, the platform delivers fluid, high-definition video playback natively within the application, entirely bypassing the need for third-party media plugins.
- **Impregnable Multi-Tenant Architecture:** The engineering of strict cryptographic security middleware and relational database constraints ensures absolute data isolation. The platform safely hosts multiple, independent agricultural entities on the same underlying infrastructure, guaranteeing that highly sensitive operational data and workplace surveillance footage remain cryptographically shielded from unauthorized access.

8.3. Future Work and System Expansion

While the current iteration of the Moo-tion platform successfully delivers a robust, production-ready vertical slice of estrus detection technology, the underlying architectural foundation was purposefully designed to accommodate extensive future scaling. The following avenues represent the most critical paths for future research, development, and commercial expansion.

8.3.1. Automated Animal Re-identification

Presently, the system operates as a sophisticated anomaly detector; it accurately alerts the human operator that *an* animal is exhibiting estrus behavior, but relies on the human to visually identify the specific animal from the provided photographic evidence. The most immediate future expansion involves fully automating this identification pipeline.

By integrating optical character recognition models alongside the primary object detection network, the system can be trained to actively scan the video feed for standard agricultural ear tags. Furthermore, advanced biometric pattern matching could be utilized to identify cattle based on their unique coat patterns (specifically for Holstein breeds). Successfully

implementing autonomous re-identification would entirely remove the human from the immediate data-entry loop, automatically logging the confirmed estrus event directly to the specific animal's historical record without requiring manual intervention.

8.3.2. Comprehensive Holistic Behavioral Analytics

The existing artificial intelligence model is highly specialized, fine-tuned exclusively to identify the physical mounting interactions indicative of estrus. However, the surveillance infrastructure is already in place to monitor the herd continuously. Future iterations of the platform will expand the neural network's classification capabilities to encompass a broader spectrum of holistic herd health analytics.

Secondary specialized models will be trained to detect subtle behavioral anomalies. This includes automated locomotion scoring to identify early-onset lameness, the monitoring of the feed bunk to track irregular eating and drinking patterns, and the detection of imminent parturition (calving distress). By expanding the recognized behavioral catalog, Moo-tion will evolve from a singular estrus-detection utility into a comprehensive, facility-wide herd health monitoring ecosystem.

8.3.3. Offline Edge Computing Infrastructure

The platform's current architecture relies on transferring lightweight, compressed data to cloud servers. While highly optimized for low bandwidth, it still requires a stable internet connection. Many rural agricultural facilities suffer from complete network blackouts. To ensure absolute operational continuity regardless of external infrastructure, future development will focus on localized edge computing.

This involves engineering custom, low-cost physical hardware nodes, compact, environmentally sealed computers equipped with dedicated neural processing units. These edge devices will be physically installed directly inside the barn facility, ingesting and processing the heavy video feeds entirely over the local area network. By executing the artificial intelligence inference locally, the system will only require an internet connection to dispatch the final, kilobyte-sized push notification, effectively immunizing the platform against rural broadband limitations.

8.3.4. Hardware Ecosystem and Facility Integration

The ultimate evolution of the Moo-tion platform involves moving beyond passive notification and interfacing directly with active physical farm infrastructure. Modern commercial dairies frequently utilize automated drafting gates and physical sorting mechanisms that route cattle into specific holding pens as they exit the milking parlor.

Future iterations of the application gateway will feature extended application programming interfaces designed to communicate securely with these proprietary sorting systems. When the vision engine definitively detects an animal in estrus, the Moo-tion backend could instantly transmit a digital signal to the drafting gate. As the specific animal passes

through the gate hours later, the physical hardware would automatically isolate her into a dedicated veterinary pen. This seamless integration of advanced software intelligence and physical mechanical sorting would represent the absolute pinnacle of automated, low-stress precision agriculture.

9. Glossary

Active Farm Environment

The specific, cryptographically isolated digital workspace currently selected by an authenticated end-user. The selection of this environment acts as the absolute primary filter for the client application, determining the scope of all subsequent data queries, hardware interactions, and interface renderings to prevent cross-tenant data exposure.

Active Learning Loop

A continuous, automated machine learning maintenance methodology. In this framework, anomalous system outputs, specifically instances where a human operator explicitly corrects a false-positive alert, are systematically archived. This human-verified dataset is subsequently utilized to periodically retrain the core neural network, thereby mitigating statistical model degradation over time and compounding system accuracy.

API (Application Programming Interface)

A set of protocols and routing definitions that allow different software subsystems (such as the mobile application and the central backend) to communicate and securely exchange data.

Asynchronous Data Gateway

A highly optimized central server architecture designed to manage thousands of concurrent input and output operations without blocking the execution threads of the primary processor. This architectural choice is vital for ensuring the instantaneous, simultaneous handling of multiple incoming network video streams and outgoing mobile application requests.

Bounding Box

A mathematically defined, two-dimensional geometric coordinate system utilized by the object detection model. It physically encapsulates a recognized biological subject within a digital video frame, establishing the precise spatial boundaries required by the system to perform subsequent spatial overlap calculations and deduplication routines.

Concept Drift

A well-documented phenomenon in deployed machine learning systems wherein the statistical properties of the target environment change unpredictably over time. In agricultural

settings, this is typically caused by shifting environmental factors such as seasonal natural lighting, barn renovations, or the introduction of new animal breeds, leading to a gradual degradation in the model's predictive accuracy if left unmaintained.

Deduplication (Spatial and Temporal)

The complex algorithmic process of mathematically evaluating and merging multiple, sequential positive neural network inferences of the exact same physical event into a single, cohesive database record. This logic is the cornerstone of the platform's usability, expressly designed to prevent notification fatigue by ensuring farm personnel receive only one alert per sustained biological incident.

Edge Computing Infrastructure

A decentralized computational architecture paradigm where heavy data processing, specifically the ingestion and artificial intelligence inference of high-definition video, occurs locally on physical hardware situated directly within the agricultural facility. This severely reduces the system's reliance on external cloud bandwidth and ensures operational continuity during rural internet blackouts.

Estrus (Heat Cycle)

The specific biological period of sexual receptivity in female cattle marks the optimal chronological window for artificial insemination. Within the context of the Moo-tion platform, this is the primary physiological event the artificial intelligence is explicitly trained to detect via the identification of sustained, physical mounting behaviors between animals.

FastAPI

A modern, highly performant web framework for building application programming interfaces with Python. It is utilized in the Moo-tion architecture to construct the central, asynchronous backend gateway.

Firebase

A comprehensive cloud development platform is utilized by the platform for handling secure user authentication, generating cryptographic identity tokens, and providing cloud object storage for visual evidence frames.

Flutter

An open-source user interface software development kit created by Google. It allows the development of natively compiled, cross-platform applications (iOS, Android, and Web) from a single unified codebase, heavily utilizing reactive state management.

Frame Sampling

A computational resource optimization technique. Rather than processing every sequential image in a thirty-frame-per-second video feed, the vision processing engine deliberately extracts, downscales, and analyzes only a mathematically calculated fraction of those frames. This preserves critical graphical processing power while maintaining acceptable biological detection fidelity.

GDPR (General Data Protection Regulation)

A comprehensive data protection law enacted by the European Union governing data privacy and user rights. The platform complies with these standards through strict multi-tenant data isolation and data minimization practices.

Inference

The real-time computational process by which a fully trained neural network analyzes new, previously unseen visual data. During inference, the model applies its internal mathematical weights to the incoming pixels to generate a statistically weighted prediction regarding the presence, geometry, and specific behavior of recognized subjects.

Intersection Over Union (IoU)

A mathematically rigorous evaluation metric utilized heavily by the processing engine's deduplication algorithm. It calculates the exact percentage of spatial overlap between two distinct geometric bounding boxes generated across consecutive temporal video frames, allowing the system to deduce if two individual detections represent the same continuous physical interaction.

JWT (JSON Web Token)

A compact, cryptographically signed token format utilized for securely transmitting verified user identity and session authorization data between the client application and the backend gateway.

KVKK (Kişisel Verilerin Korunması Kanunu)

The Turkish Personal Data Protection Law. The application complies with this regulatory framework by ensuring explicit consent mechanisms, secure data handling, and strict limits on the retention of workplace surveillance footage.

Motion-JPEG (MJPEG)

A specific video compression format wherein each video frame of a digital sequence is compressed separately as a distinct, standalone image. The Moo-tion platform utilizes this

format to seamlessly transcode raw network camera streams, bypassing the native streaming incompatibilities prevalent across modern web browsers and mobile operating systems.

Multi-Tenancy

A foundational software architecture principle wherein a single, global instance of a software application securely serves multiple, entirely independent agricultural organizations. This architecture maximizes infrastructure efficiency while enforcing absolute cryptographic data isolation between the independent organizations.

PostgreSQL

A highly advanced, open-source relational database management system utilized by the platform to securely store all structured operational data, multi-tenant mappings, and chronological herd records.

Reactive State Management

A declarative software engineering paradigm is utilized extensively within the client application. In this model, the user interface does not actively request data updates; rather, it automatically and instantaneously redraws its visual elements in response to underlying mutations within a localized, strictly managed cache of device memory.

Real-Time Streaming Protocol (RTSP)

An international telecommunications standard explicitly designed for controlling streaming media servers. The platform strictly adheres to this protocol for its video ingestion pipeline to ensure universal, plug-and-play interoperability with the vast majority of commercial surveillance hardware available on the global market.

Role-Based Access Control (RBAC)

A rigorous cybersecurity mechanism is implemented within the central gateway. It restricts system modifications, sensitive data retrieval, and destructive administrative actions strictly to users holding specifically verified, hierarchical cryptographic roles (such as Facility Owner or Operational Manager) within the active farm environment.

State Machine (Reproductive)

An event-driven algorithmic engine housed within the central backend. It continuously evaluates an animal's current biological status based on a chronological history of logged veterinary events, automatically transitioning the animal through sequential phases of the reproductive lifecycle and generating predictive future timelines.

Transfer Learning

A highly efficient machine learning technique where a neural network originally developed and trained for a vast, generic visual task is subsequently utilized as the foundational starting point for a model focused on a highly specific task (e.g., detecting bovine estrus). This methodology drastically reduces the required computational time and raw data volume necessary to achieve commercial-grade accuracy.

YOLO (You Only Look Once)

A state-of-the-art, real-time object detection algorithm system is utilized by the vision processing engine. It is renowned for its immense processing speed, enabling the system to evaluate high-definition video frames and classify biological behaviors with minimal latency.

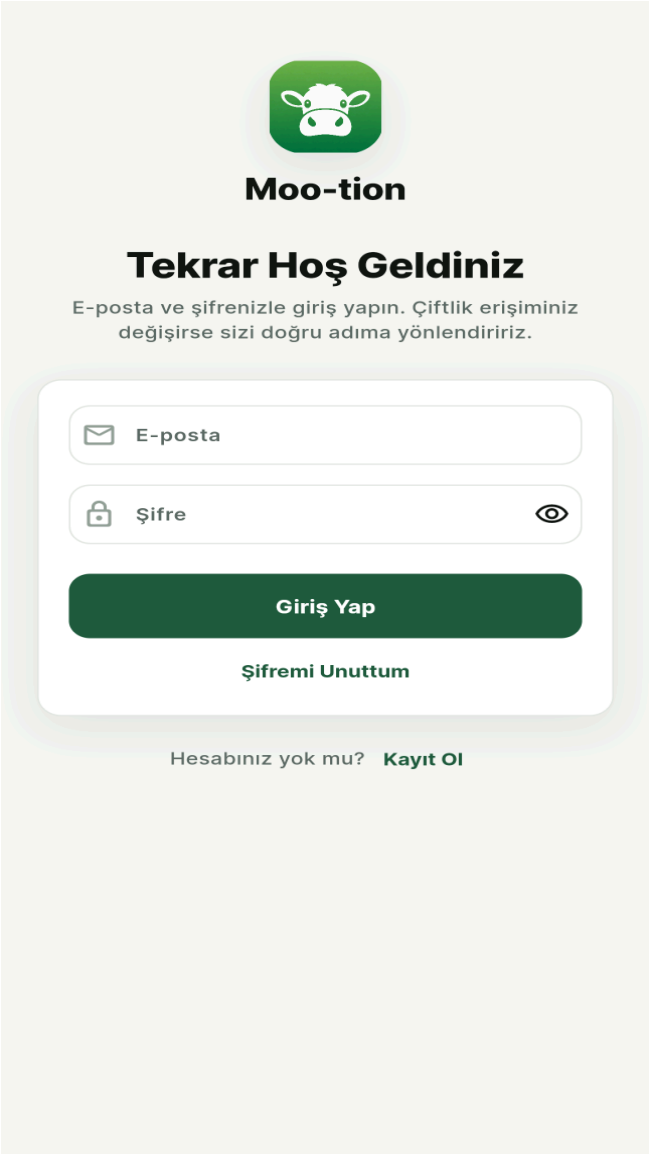
Moo-tion User Manual

1 - Logging into Moo-tion

1.1 - Sign-up

The Moo-tion application requires a user account to ensure farm data remains secure and accessible only to authorized personnel. From the initial launch screen, new users can navigate to the registration page by tapping the "Kayıt Ol" button. This page presents a straightforward form requesting the user's name, a valid email address, and a secure password, along with a password confirmation field. It is highly recommended to use an email address that is easily accessible, as this will be the primary point of contact for account verification and critical farm alerts.

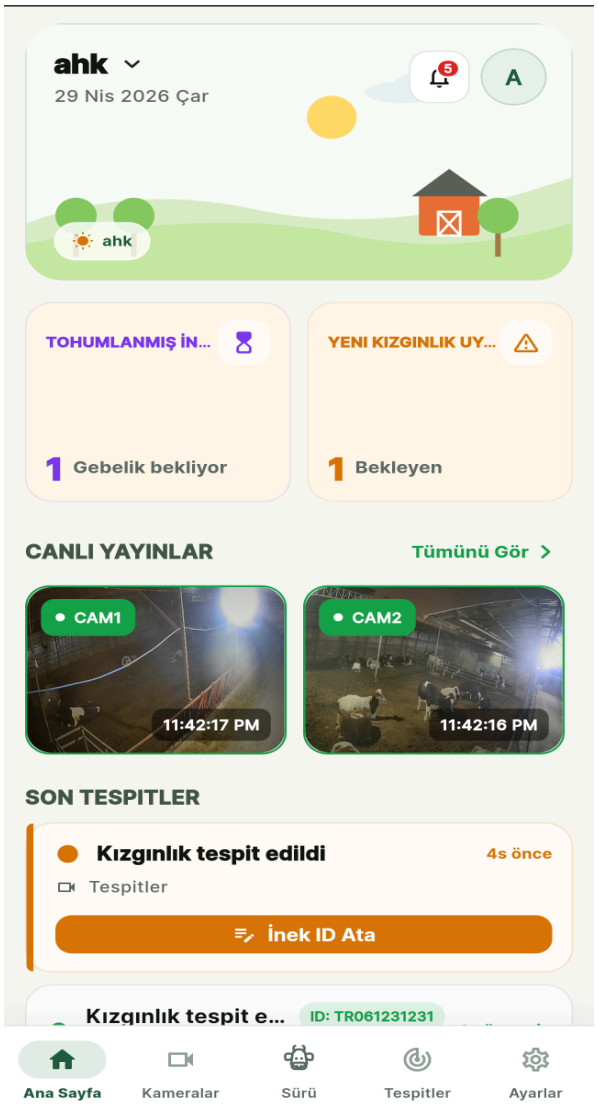
1.2 - Sign-in



The sign-in page is designed to quickly get farm owners and caretakers into their operational dashboard. Users simply need to input the email address and password associated with their account and tap "Giriş Yap". For convenience, if a user happens to forget their credentials, a "Şifremi Unuttum" link is available to initiate the password recovery process. Once successfully authenticated, the application will seamlessly route the user to their designated farm dashboard.

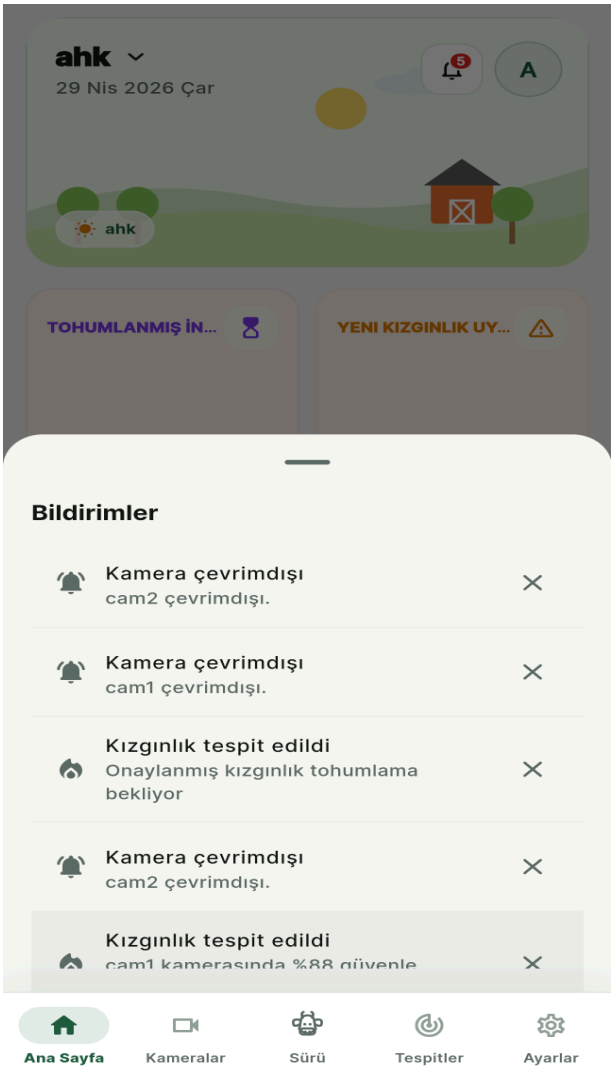
2 - Moo-tion Homepage and Notifications

2.1 - Home Dashboard



Upon logging in, users are greeted by the main dashboard, which serves as a high-level operational snapshot of the farm. The top of the screen displays summary cards highlighting active reminders, such as inseminated cows or pending daily actions. Just below, the "Canlı Yayınlar" section provides live camera thumbnails, allowing managers to quickly verify the visual environment of the barn. Additionally, a "Son Tespitler" section highlights recent AI detections requiring immediate attention. A fixed bottom navigation bar provides quick access to the Home, Cameras, Herd, Detections, and Settings pages.

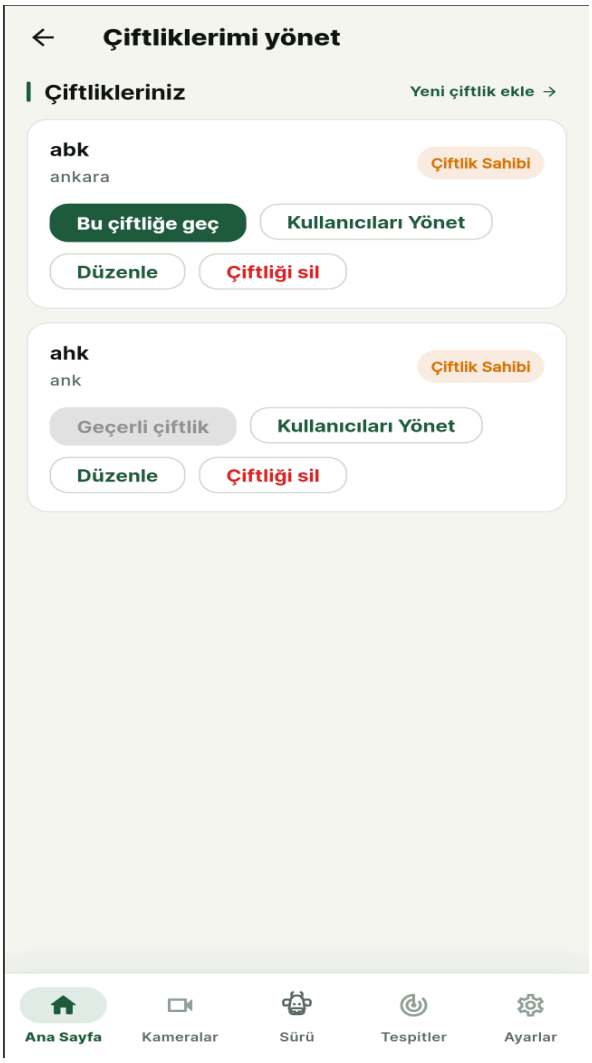
2.2 - Notifications Panel



To keep caretakers informed of real-time events, the application features an accessible notification drawer. This panel lists recent system and environmental events, such as a camera going offline or a new estrus (kızgınlık) detection. It is best practice to review and clear acknowledged items regularly, keeping the operator queue focused on unaddressed tasks and ensuring infrastructure reliability.

3 - Farm Management

3.1 - Managing Existing Farms



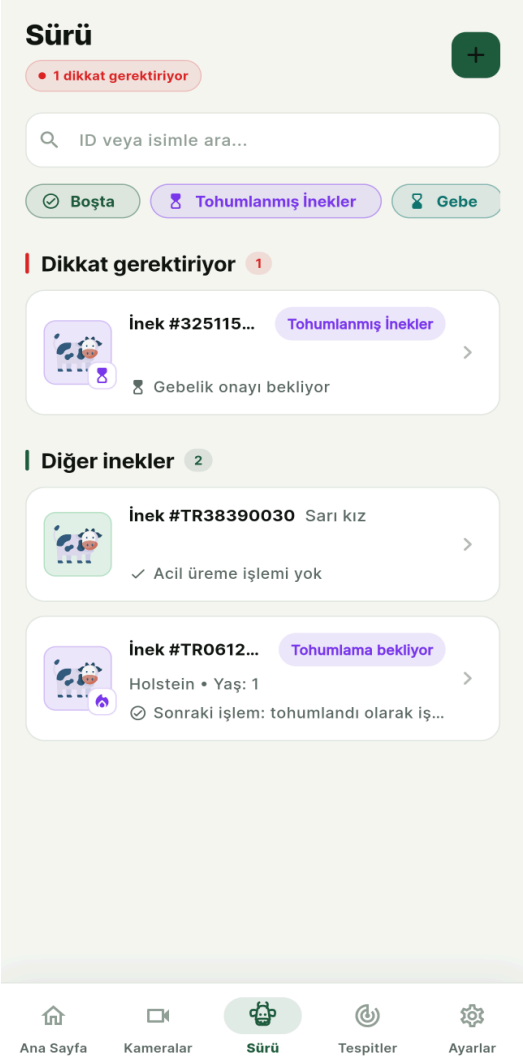
The farm management screen provides farm owners with a comprehensive view of all the facilities they are authorized to oversee. From this centralized hub, users can switch their active context by tapping "Bu çiftliğe geç," ensuring that the data displayed throughout the app strictly pertains to the selected location. This page also serves as the gateway for managing user roles and editing or deleting farm entries based on access permissions.

3.2 - Creating a New Farm

When expanding operations, administrators can easily register new locations in the system. By selecting "Yeni çiftlik ekle" from the farm management menu, users are prompted to fill out essential details, including the farm's name, the city it is located in, and an optional descriptive text. Creating a distinct farm entry ensures that all subsequent data, such as cow profiles, camera feeds, and detections, remain neatly isolated within that specific operational scope.

4 - Herd and Cow Lifecycle Operations

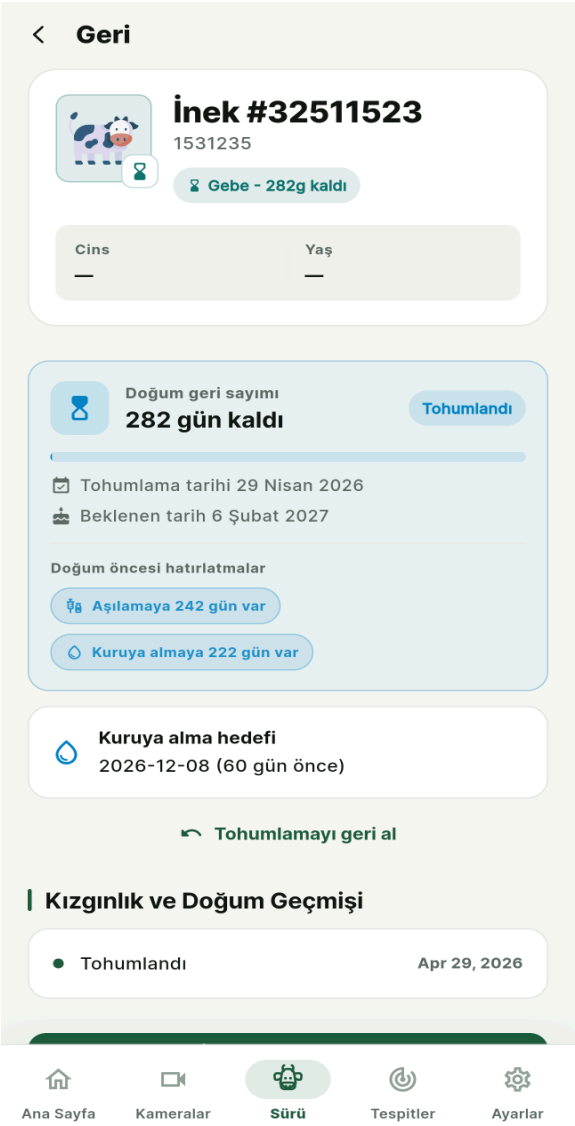
4.1 - Herd Overview



The "Sürü" tab acts as the primary operational queue for animal tracking and management. This screen lists the entire registered herd and provides intuitive tools to search for specific animals by their ID or name. Users can filter the herd based on current lifecycle statuses, such as pregnant (gebe), inseminated (tohumlanmış), or empty (boşta). Furthermore, a priority grouping feature flags cows that require immediate attention, helping caretakers organize their daily workflow efficiently.

4.2 - Adding and Managing Cow Profiles

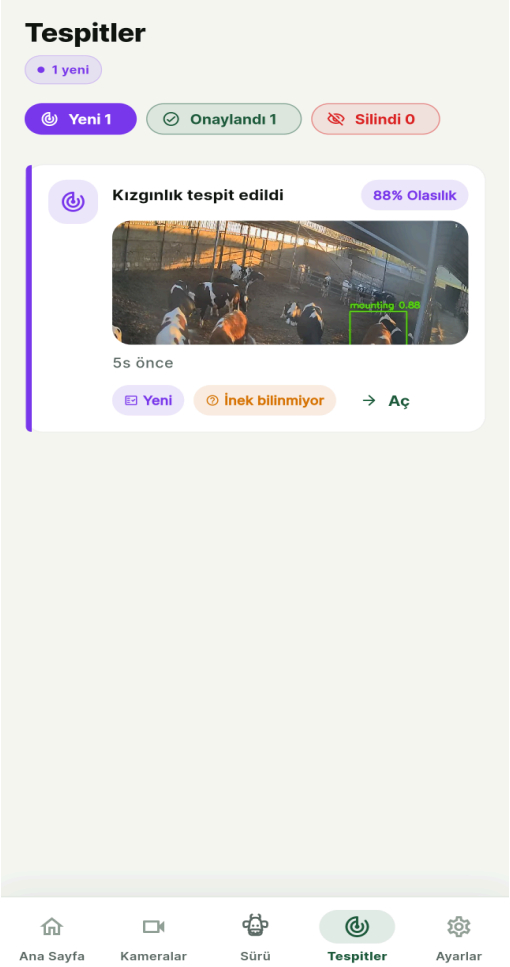
To maintain an accurate digital representation of the herd, users can add new animals by tapping the "+" icon on the herd page. This opens a form requiring a unique Tag ID, alongside optional details like name, breed, birth date, and default dry-off days.



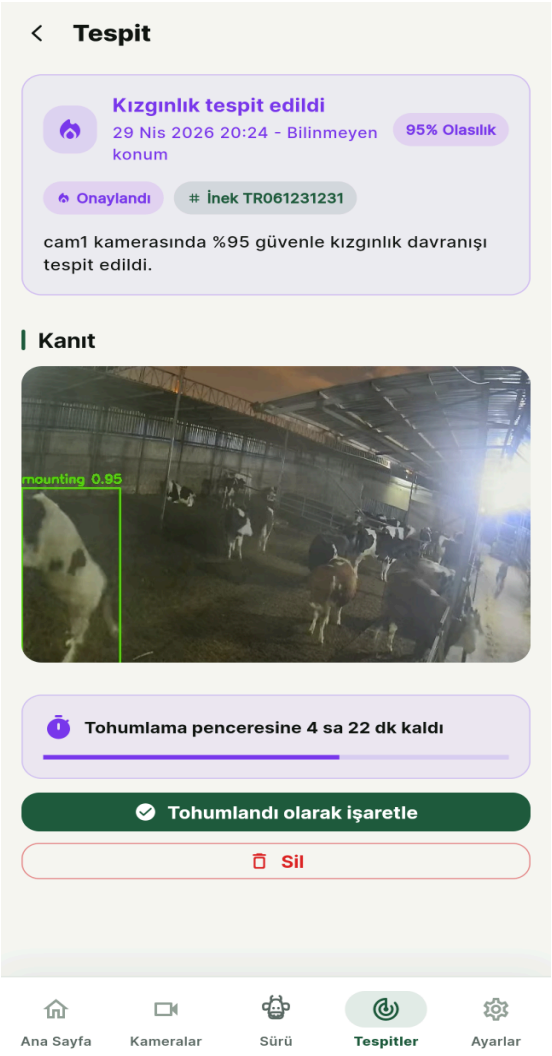
Once an animal is registered, opening its profile reveals a comprehensive reproductive and health timeline. The profile page tracks the cow's current status, displays a countdown to expected calving, and schedules pre-birth reminders like vaccinations. It also houses a complete event history, making it an invaluable tool for auditing prior actions and confirming veterinary timelines.

5 - Detection Management (Tespitler)

5.1 - Reviewing New Detections

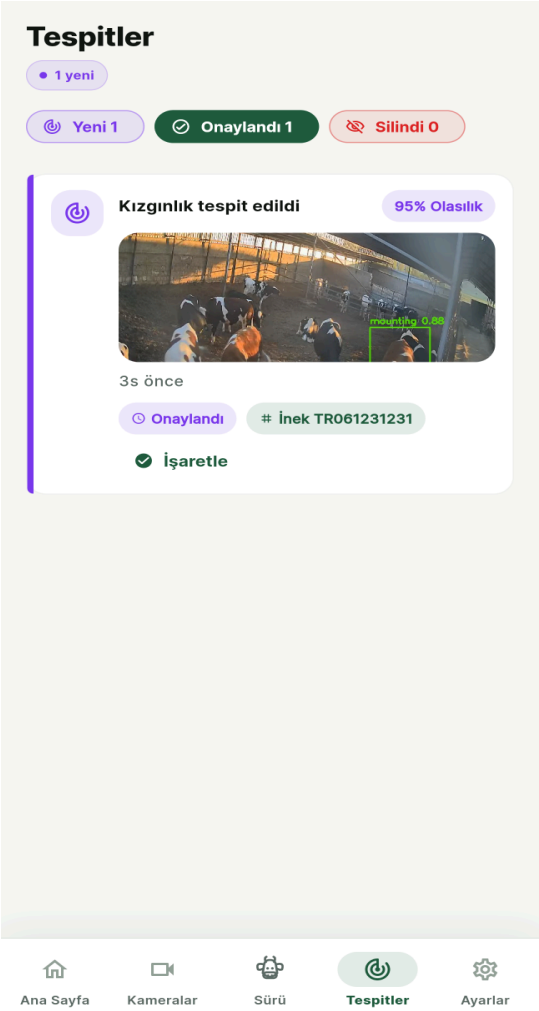


The core of the Moo-tion app is its AI-driven detection system, accessible via the "Tespitler" tab. New events automatically populate in the "Yeni" category, presenting users with actionable cards. Each card displays the event type, the AI's confidence score, a thumbnail image of the evidence, and the precise time of occurrence.

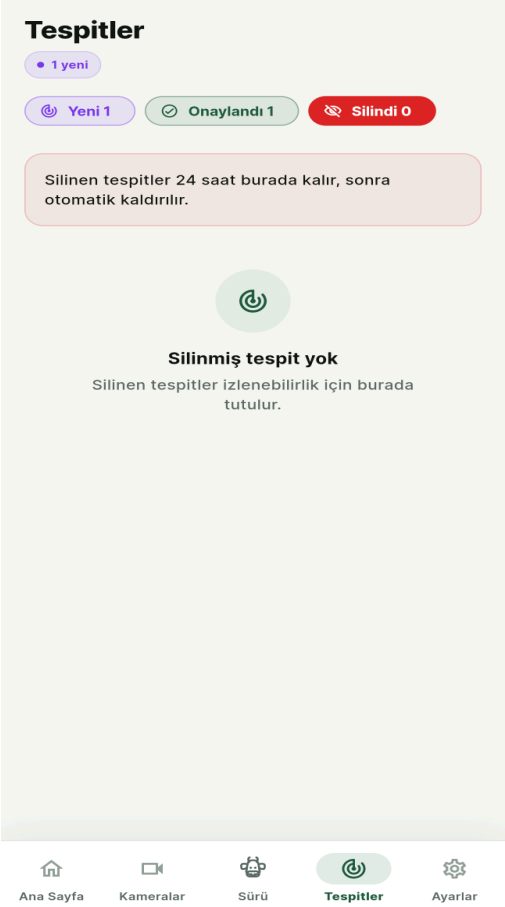


When a user opens a detection, they can evaluate the operational validity of the image and choose to either manually link the event to a specific cow profile by marking it as inseminated ("Tohumlandı olarak işaretle") or delete the record if it is deemed a false alarm.

5.2 - Approved and Deleted Queues



To maintain a clean workflow while preserving historical data, processed detections are sorted into dedicated queues. The "Onaylandı" (Approved) queue houses all events that have been accepted and linked to a cow's lifecycle tracking.



Conversely, the "Silindi" (Deleted) queue acts as a temporary holding area for rejected events, typically retaining them for 24 hours before permanent removal, allowing users to reverse accidental deletions.

6 - Camera Management

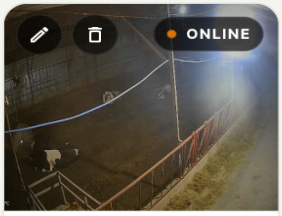
6.1 - Camera Health and Monitoring

Continuous visual monitoring is maintained through the Kameralar page, which provides a detailed list of all connected surveillance units. This interface displays individual camera cards featuring live preview thumbnails and a clear indicator of their online or offline status. As part of a daily routine, users are encouraged to verify the online count and inspect these previews to quickly identify and resolve any disconnected cameras, thereby preventing missed AI detections.

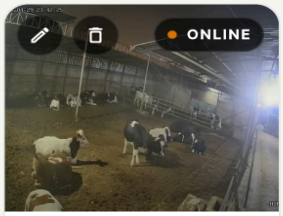
Kameralar

2 / 2 çevrimiçi

Kamera Ekle
Yeni bir yayın bağla



cam1
Açıklama yok



cam2
Açıklama yok

Ana Sayfa | Kameralar | Sürü | Tespitler | Ayarlar

Kameralar

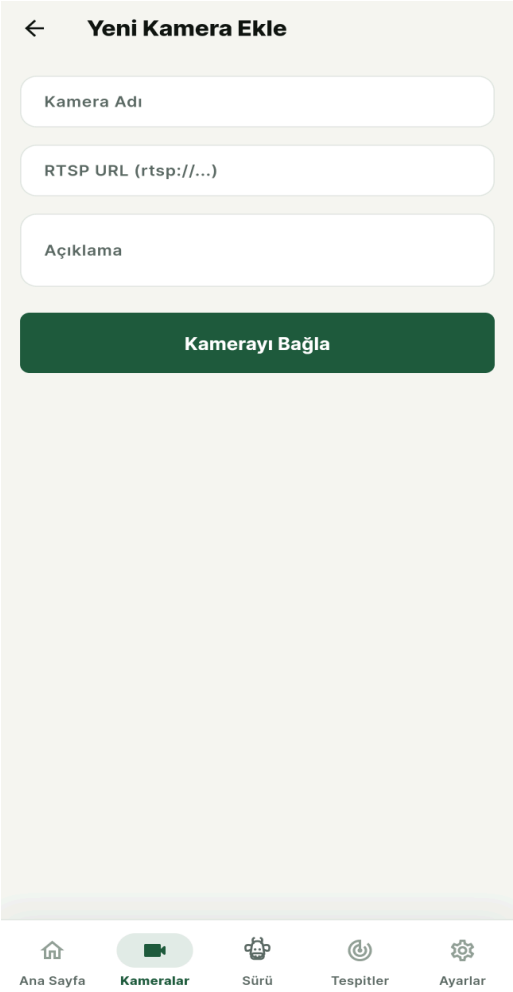
0 / 0 çevrimiçi

Kamera bulunamadı
Kamera Ekle

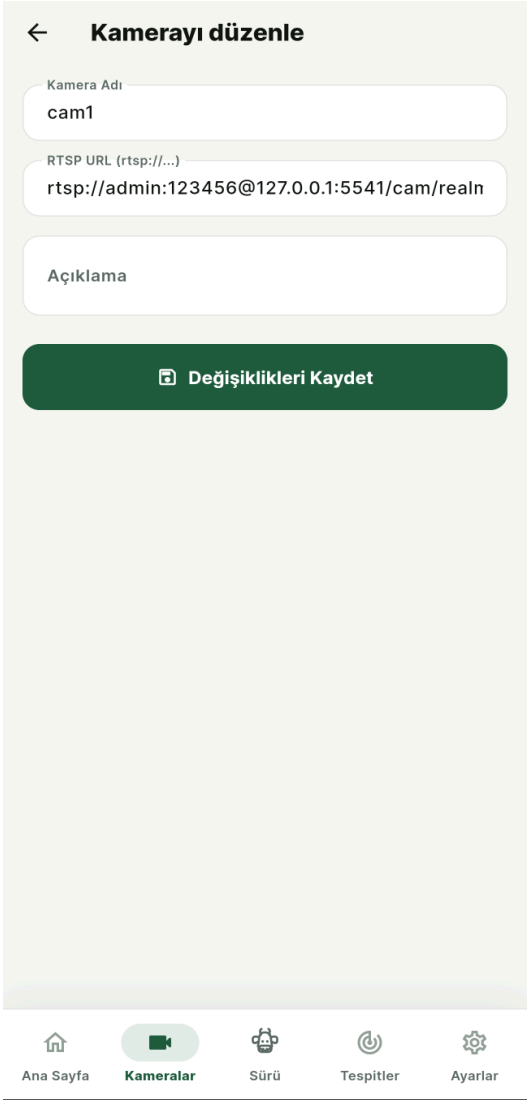
Kamera Ekle
Yeni bir yayın bağla

Ana Sayfa | Kameralar | Sürü | Tespitler | Ayarlar

6.2 - Connecting and Editing Cameras



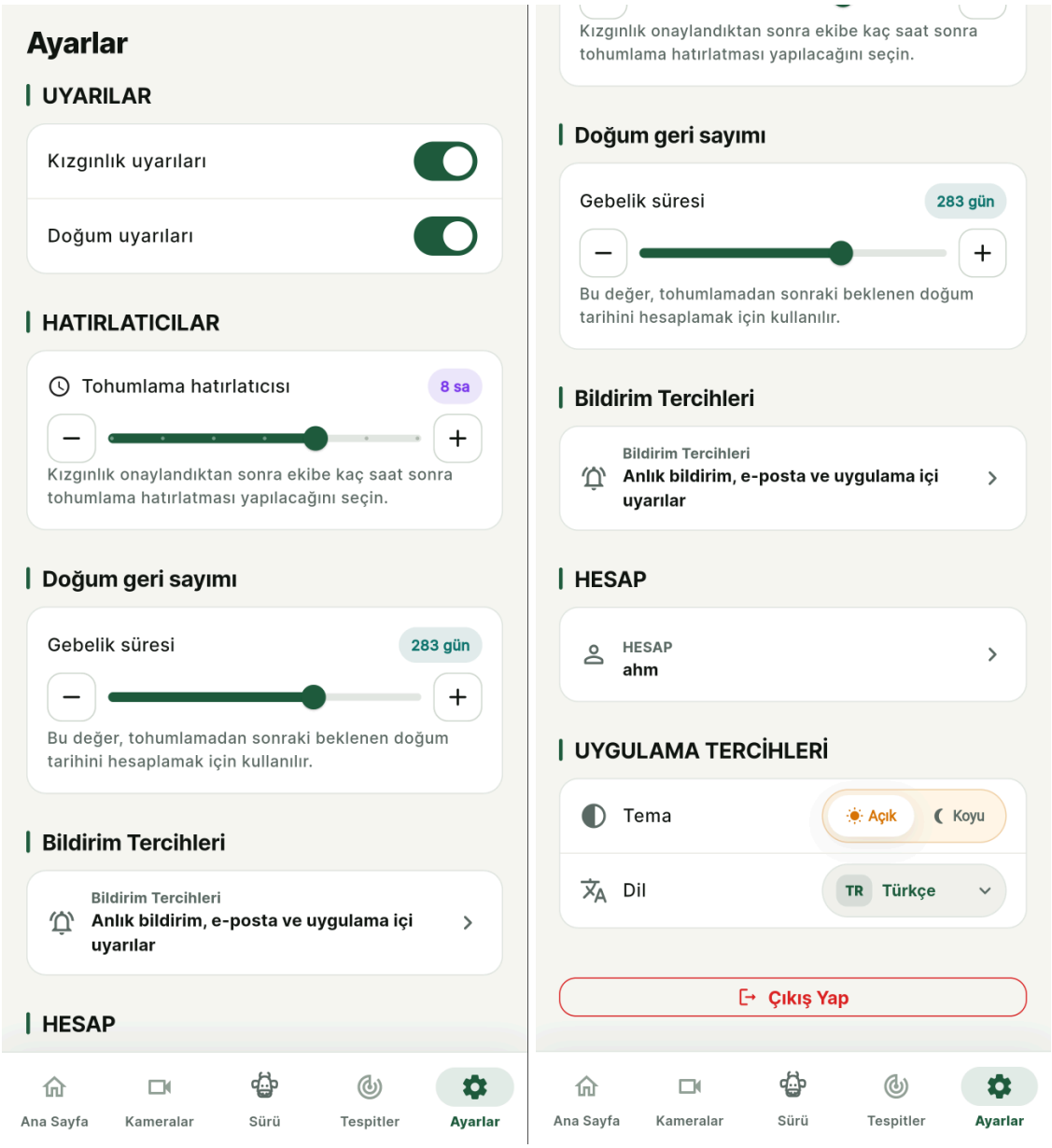
Expanding or updating the visual network is handled directly within the app. By tapping "Kamera Ekle," users can connect new hardware by inputting a camera name, an RTSP URL, and an optional description.



For existing setups that require maintenance—such as a changed IP address or an updated location name—users can simply open the specific camera card, tap the edit icon, and save their changes to instantly restore the video feed.

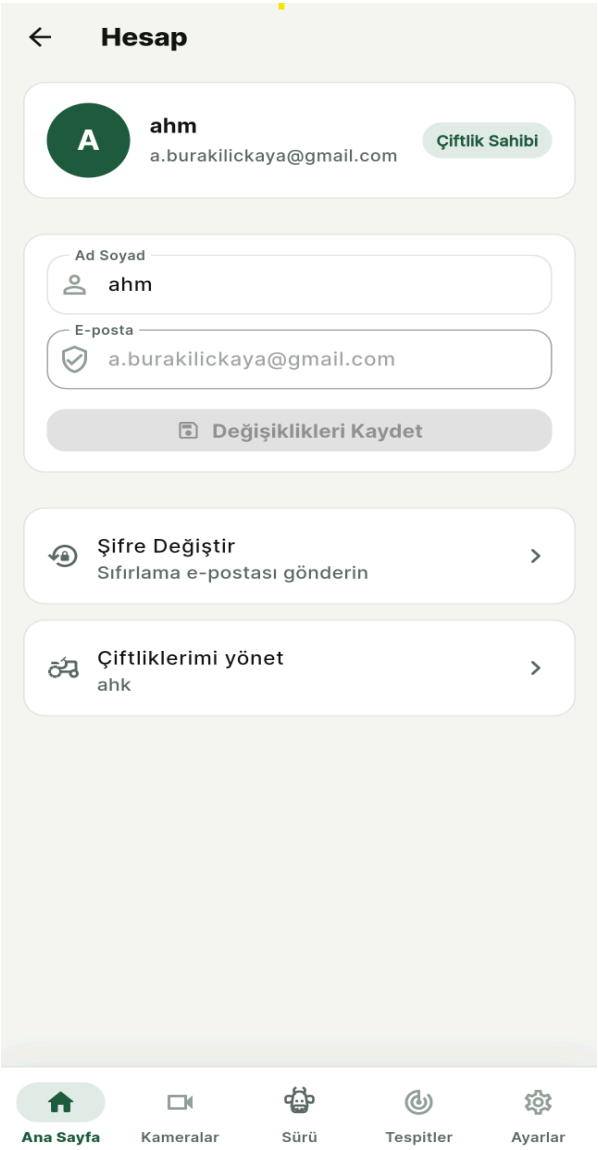
7 - Settings, Account, and Preferences

7.1 - General System Settings



The Ayarlar (Settings) menu is designed to let managers customize the app's behavior to match their specific operational guidelines. Here, users can toggle specific alerts on or off, such as estrus warnings or offline camera notifications. It also provides granular control over operational parameters, allowing users to adjust insemination reminder windows, define average pregnancy durations, and select their preferred language and interface theme.

7.2 - Account Profile and Notification Channels



Personalization extends to the individual user profile and their communication preferences. The account settings screen allows users to view and update their profile details, trigger a password reset, and quickly navigate back to farm management.

More importantly, the notification preferences section gives users the power to determine exactly how they receive alerts. Users can configure delivery methods on an event-by-event basis, choosing between instant push notifications, email summaries, or strictly in-app alerts, ensuring the right people get the right message at the right time.

10. References

[1] J. K. Roelofs, F. J. C. M. van Eerdenburg, N. M. Soede, and B. Kemp, "Various behavioral signs of estrus and their relationship with time of ovulation in dairy cattle," *Theriogenology*, vol. 63, no. 5, pp. 1366-1377, Mar. 2005.

[2] S. J. LeBlanc, "Assessing the efficacy of visual observation, pedometry, and automated activity monitoring for the detection of estrus in dairy cows," *Journal of Dairy Science*, vol. 91, no. 8, pp. 3200-3209, Aug. 2008.

[3] M. A. Crowe, M. Hostens, and G. Opsomer, "Reproductive management in dairy cows - the future," *Irish Veterinary Journal*, vol. 71, no. 1, p. 1, Jan. 2018.

[4] D. Berckmans, "Precision livestock farming technologies for welfare management in intensive livestock systems," *Revue Scientifique et Technique (International Office of Epizootics)*, vol. 33, no. 1, pp. 189-196, Apr. 2014.

[5] A. Rutten, C. Velthuis, W. Steeneveld, and H. Hogeveen, "Economic benefit of pelvimeters, activity meters, and pedometers for estrus detection in dairy cattle," *Journal of Dairy Science*, vol. 97, no. 6, pp. 3550-3565, Jun. 2014.

[6] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 779-788.

[7] A. Bochkovskiy, C. Y. Wang, and H. Y. M. Liao, "YOLOv4: Optimal Speed and Accuracy of Object Detection," *arXiv preprint arXiv:2004.10934*, 2020.

[8] W. Wang, Y. Zhang, H. Li, and P. Li, "Deep learning-based automated behavioral analysis of livestock from video streams," *Computers and Electronics in Agriculture*, vol. 187, p. 106263, Aug. 2021.

[9] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 6, pp. 1137-1149, Jun. 2017.

[10] T. Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal Loss for Dense Object Detection," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, Venice, Italy, 2017, pp. 2980-2988.

[11] M. Qiao, L. Liu, Z. He, and Y. Wang, "Computer vision tracking and identification of cattle using Intersection over Union algorithms in crowded barn environments," *Precision Agriculture*, vol. 22, no. 3, pp. 889-906, 2021.

[12] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, USA: Addison-Wesley Longman Publishing Co., Inc., 2002.

[13] N. Marz and J. Warren, *Big Data: Principles and best practices of scalable realtime data systems*. Shelter Island, NY, USA: Manning Publications Co., 2015.

[14] D. Crockford, *JavaScript: The Good Parts*. Sebastopol, CA, USA: O'Reilly Media, Inc., 2008. (Utilized for foundational principles in reactive client-side architecture).

[15] C. Richardson, *Microservices Patterns: With examples in Java*. Shelter Island, NY, USA: Manning Publications Co., 2018. (Utilized for principles of decentralized data management and worker process isolation).

[16] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley, 1994. (Utilized for structural implementation of the Repository and Dependency Injection patterns).

[17] H. Schulzrinne, A. Rao, and R. Lanphier, "Real Time Streaming Protocol (RTSP)," Internet Engineering Task Force (IETF), Request for Comments: 2326, Apr. 1998. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc2326

[18] International Organization for Standardization, "ISO/IEC 25010: Systems and software Quality Requirements and Evaluation (SQuaRE) System and software quality models," Geneva, Switzerland, 2023.

[19] D. Hardt, Ed., "The OAuth 2.0 Authorization Framework," Internet Engineering Task Force (IETF), Request for Comments: 6749, Oct. 2012. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc6749

[20] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," Internet Engineering Task Force (IETF), Request for Comments: 7519, May 2015. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc7519

[21] Association for Computing Machinery, "ACM Code of Ethics and Professional Conduct," Jun. 2018. [Online]. Available: https://www.acm.org/code-of-ethics

[22] Republic of Turkey, "Kişisel Verilerin Korunması Kanunu [Personal Data Protection Law]," *Resmî Gazete* [Official Gazette], no. 29677, Apr. 7, 2016. [Online]. Available: https://www.resmigazete.gov.tr/eskiler/2016/04/20160407-1.htm

[23] European Parliament and Council of the European Union, "Regulation (EU) 2016/679 (General Data Protection Regulation)," *Official Journal of the European Union*, L 119/1, May 2016. [Online]. Available: https://eur-lex.europa.eu/eli/reg/2016/679/oj

[24] Institute of Electrical and Electronics Engineers, "IEEE Code of Ethics," IEEE Policies, Section 7 - Professional Activities, Jun. 2020. [Online]. Available: https://www.ieee.org/about/corporate/governance/p7-8.html